

Merchant Sales & Voucher Intelligence Dashboard

BI Developer — Second-Round Practical Task

Microsoft Fabric · Power BI · dbt · SQL · Python ML

Deliverable	Detail
Reporting period	1 January 2026 – 31 July 2026 (212 days)
Data volume	120,969 vouchers · 26,500 daily sales rows · 1,363 tickets · 25 merchants
Architecture	Fabric medallion (bronze / silver / gold) → Kimball star schema → Power BI (import; Direct Lake once hosted)
Transformation	dbt: 18 models, 1 snapshot (SCD2), 132 tests, 4 seeds, 3 exposures
Orchestration	Data Factory pipeline, metadata-driven, daily 02:00 SAST
Semantic model	6 dimensions · 4 facts · 1 analytics mart · 31+ validated DAX measures
AI extension	5 models: anomaly detection, propensity, regression, forecasting, segmentation
Quality gates	14 warehouse tests · 132 dbt tests (153 pass on build, 0 warn, 0 error) · 7 SCD2 assertions · 27/28 reconciliation checks

Anthony Apollis

11 August 2026

Contents

Contents	2
1. Executive summary.....	5
Headline figures.....	5
The four findings that matter	5
2. Solution architecture	7
Naming convention	7
Orchestration — Azure Data Factory / Fabric Pipelines.....	7
Fabric cost control and platform enhancements	8
Warehouse and Lakehouse enhancements applied.....	8
Retry policy	8
3. Data model	10
What the drop actually supplies.....	10
Source column lineage, and what the supplied dictionary covers	10
The four supplied tables on their own	12
Why fourteen tables when the README suggests five	13
Before and after	15
Tables.....	18
Entity relationships.....	18
Normalisation, keys and the Kimball position	19
Modelling decisions worth defending	20
4. Transformation layer	22
Business rules, defined once	22
SQL report pack — the queries behind the numbers.....	22
A finding about the supplied targets	24
What dbt is used for, and what it earned.....	25
dbt project	25
Data governance.....	27
Governance as an operating model	27
Platform choices — storage modes, fallback and mirroring	28
Data dictionary	30
Worked example — a wrong number that looked right	36

Cross-implementation reconciliation	37
5. Semantic model and report	38
DAX measure library	38
Report pages	38
Conditional formatting	39
6. AI and machine learning extension	40
Anomaly detection — and how it was validated	40
Reporting a weak metric honestly	40
Segmentation — choosing k on business grounds	41
7. Business questions answered	42
Q1. Which merchants generate the highest sales value and transaction volume?	42
Q2. Which voucher type has the highest redemption rate?	42
Q3. Which region shows declining sales or transaction behaviour?	42
Q4. Are support ticket volumes, priority or long resolution times associated with weaker merchant performance?	43
Q5. Which merchants should management focus on first, and why?	43
8. Reconciliation and financial controls	45
9. Merchant value, attrition risk and fraud signals	45
Geographic intelligence	46
Fraud detection AND prevention	46
Fraud controls — what the data supports, tested not assumed	47
8. Recommendations	48
9. Data quality, assumptions and limitations	49
Data quality findings	49
Assumptions	49
Limitations	49
Appendix A. Evidence	51
Part 1 — Fabric workspace	51
Part 2 — dbt connected	51
Part 5 — cost control	53
Part 6 — build verification	54
Appendix B. Fabric connection evidence	55
Extending this to a production reporting environment	56
Exception-based reconciliation	56
A governed layer for AI and self-service consumption	56

Scaling to additional regions.....	56
10. Deliverables and how to run	57
Reproducing the results.....	57
Appendix F. End-to-end Fabric process captures	58
1. Source ZIP on the PC.....	58
2. PC package lands in OneLake	59
3. Fabric notebook unpacks and ingests	60
4. Extracted source files verified in OneLake.....	61
5. Data Factory executes the medallion chain	62
6. Bronze preserves the delivery	63
7. Silver conforms and cleans	64
8. Gold publishes reporting tables.....	65
9. Validation gate reconciles all layers	66
10. Gold serves the reporting query	67

1. Executive summary

This solution delivers a production-minded analytics stack over the supplied merchant voucher data: a Fabric medallion pipeline feeding a Kimball star schema, a dbt transformation layer with a 69-test suite, a four-page Power BI report, and five machine-learning models covering anomaly detection through to forecasting. Every number in every artefact traces to a single gold layer.

Headline figures

Metric	Value	Note
Total sales	R65.5m	510,127 transactions, avg basket R128
Redemption rate	84.2%	101,843 of 120,969 vouchers
Avg days to redeem	3.58	median 2 days, max 54
Delayed redemption rate	13.9%	redeemed after > 7 days
Outstanding liability	R3.5m	unredeemed voucher value
Avg resolution hours	23.7	median 16.4h — right-skewed
SLA breach rate	26.3%	94.7% of breaches sit on High/Critical
Open ticket backlog	239	Open, Escalated or Pending Merchant

The four findings that matter

1. One merchant is failing, and it is an operational failure, not a demand problem. Umhlanga Value Mart's July sales fell 42.5% against its own prior three-month average while support tickets rose 693% in the same month. The two moving together points to a service breakdown causing commercial damage — which means it is plausibly recoverable. Annualised revenue at risk is **R571,518**.

2. The SLA policy is configured backwards. Critical tickets are given a 12-hour target but take 52.7 hours on average — a 98.3% breach rate. Low-priority tickets are given 48 hours and take 11.3 — a 0.2% breach rate. Because the ladder runs opposite to the actual workload, **94.7% of all 358 breaches land on High and Critical**. The reported 26% breach rate is measuring a policy misconfiguration, not team performance.

3. Eastern Cape is the one declining region, on four independent signals. It is the only region that peaked before July, sits 9.8% below its own peak while every other region is at its peak, has the flattest trend slope (+2.0% of average monthly sales vs +4.4% to +8.5%), and fell 12.2% in June against +1.4% to +3.8% elsewhere. Three of the five Critical/Watch merchants sit there.

4. Operational friction does NOT predict merchant performance — and the obvious analysis says it does. Tickets per 1,000 transactions correlates with target attainment at $r = -0.56$, which looks conclusive. It is confounded: the ratio is size-dependent ($r = -0.83$ against log total sales), so small merchants score badly purely because the denominator is small. Controlling for size, the partial correlation collapses to $r = -0.20$. Reporting the raw correlation would have been a confounded finding presented as a causal one.

5. The CRM 'At Risk' flag has zero overlap with actual deterioration. Both merchants flagged At Risk are **growing** and score Healthy (61.7 and 68.3). Every merchant in genuine decline — including the one losing R571,518 annualised — is flagged 'Active'. **R1,079,726** of at-risk revenue sits in merchants the CRM considers fine. The

existing manual flag is not detecting the problem it exists to detect, which is the clearest argument for a computed health score.

6. The open-ticket backlog is two problems, not one. Of 239 open tickets, **146 are awaiting us** (Open + Escalated) and **93 are awaiting the customer** (Pending Merchant). Those need entirely different remediation, and a single '239 open' figure hides the distinction — which is why `dim_ticket_status` carries an explicit ownership attribute rather than just an `is_open` flag.

Where the real operational signal is

Ticket spikes matter at the level of individual events, not as a portfolio rule. Durban Cash Hub's tickets rose 780% in June while sales GREW 8.2% — a service problem at a healthy account. Umhlanga Value Mart's rose 693% while sales FELL 42.5% — a failing account. Identical operational signal, opposite commercial diagnosis. The alerting logic in this solution therefore pairs ticket movement with sales movement rather than triggering on either alone.

2. Solution architecture

The stack follows the Fabric medallion pattern. Each layer has one responsibility, and the boundaries between them are where the quality controls sit.

Layer	Responsibility	Contents
Landing	Files as delivered	4 CSV files in OneLake Files/landing
Bronze	Land raw, add lineage only. No business logic whatsoever.	4 Delta tables + _batch_id, _ingested_at, _source_file
Silver	Type, trim, deduplicate at the declared grain, conform, apply business rules	4 Delta tables, partitioned by month, Z-ORDERed on merchant + date
Gold	Kimball star schema for consumption	6 dimensions, 4 facts, 1 analytics mart, 2 ML output tables
Consumption	Report, workbook, models	Power BI (import today, Direct Lake once hosted), Excel pack, Fabric notebooks, Copilot/Q&A

Naming convention

Applied consistently so an unfamiliar engineer can predict a table name: bronze_<source>, silver_<entity>, dim_<entity>, fct_<business_process>, mart_<consumer>. Surrogate keys are <entity>_key, natural keys are <entity>_id. Measures are named in business language, not warehouse language — 'Redemption Rate %', not 'pct_redeemed_calc'.

Orchestration — Azure Data Factory / Fabric Pipelines

Three decisions in the pipeline design are worth stating explicitly, because each one exists to prevent a specific failure that occurs in production BI.

Metadata-driven ingest. A SourceFiles parameter array carries one object per source; a ForEach iterates it. Adding a fifth source is a parameter change, not a pipeline edit — which matters because pipeline edits require deployment and regression testing, while parameter changes do not. It also guarantees all four copy activities behave identically.

The data-quality gate sits BETWEEN silver and gold. This is the most important decision in the pipeline. The dbt test suite runs against silver, and gold is only rebuilt if every test passes. If the gate fails, the pipeline stops, alerts, and leaves the previous good gold layer in place — so the report keeps showing yesterday's correct numbers rather than today's wrong ones. Loading gold first and testing afterwards means the report has already published bad figures by the time anyone reads the alert. Stale-but-correct beats fresh-but-wrong: an executive working from a day-old number makes a slightly late decision; one working from a wrong number makes a wrong decision.

Row-count validation catches the failure that does not throw. A truncated or empty source file does not raise an error. The copy succeeds, the transformation succeeds, and the dashboard shows zero — which looks exactly like a genuinely bad trading day. A per-source minimum row count turns that silent failure into a loud one. Thresholds sit well below current volumes so normal fluctuation does not trip them, but far above zero so truncation cannot pass.

Fabric cost control and platform enhancements

A Fabric capacity does not warn, throttle or stop on its own. A scheduled pipeline fires nightly for the full 60 days whether anyone is watching or not, and if a paid capacity is later attached the same schedules run against it. Cost control is therefore built into the artefacts rather than left as an operating instruction.

Measure	Implementation	Saving or risk avoided
Trigger ships disabled	TR_Daily_0200_SAST has runtimeState 'Stopped' and a hard endTime of 2026-10-03	~60 unattended pipeline runs, each spanning a Copy activity, four notebooks and a semantic model refresh
Kill switch	fabric_cost_guard.py --check / --stop / --nuke, using the az login token	Disables every schedule and cancels in-flight runs in one command
Automated watch	Two scheduled tasks: weekly escalating check, plus a one-time alert on 2026-10-03	Removes reliance on remembering the expiry date
Direct Lake as the production target	The semantic model reads Delta files directly; no import refresh	No duplicated storage, and no scheduled dataset refresh consuming capacity units
Ephemeral intermediate models	int_merchant_monthly and int_merchant_momentum are inlined as CTEs	Two fewer materialised tables, and no staleness between them and their consumers
Incremental fact load	fct_merchant_sales merges on a deterministic key with a 3-day lookback	A daily run rewrites three days, not 26,500 rows
Local development target	The dev target runs the identical models with no cloud cost	Development and testing consume zero capacity; only promotion touches Fabric

Warehouse and Lakehouse enhancements applied

Enhancement	Why
Z-ORDER on merchant_id + date in silver	Direct Lake reads Delta files directly, so file layout IS query performance. The report filters on merchant and period constantly.
Partition silver by year_month	Partition pruning is negligible at 26,500 rows but the pattern must be correct before volume arrives, not retrofitted after.
NOT ENFORCED primary and foreign keys	Fabric Warehouse does not enforce constraints but the optimiser uses them for better plans — and they document the model for the next engineer.
Bronze as VARCHAR throughout	Typing is silver's job. A cast applied at ingest is a transformation nobody can see or test, and it destroys the evidence of what the source actually sent.
Surrogate keys generated deterministically	The same natural key always yields the same surrogate, so a full rebuild cannot fork a dimension — which is what makes the pipeline idempotent.
Adapter-dispatched SQL macros	The same models target both the local development engine and the Fabric Warehouse. T-SQL has no boolean column type and rejects positional GROUP BY.

Retry policy

Activity	Retries	Rationale
Copy to Bronze	3	Transient storage/network faults are common and genuinely self-healing
Transform to Silver	1	Largely deterministic; one retry covers a capacity blip
Data Quality Gate	0	A failing test is a real defect. Retrying delays the alert and risks a flaky pass
Build Gold	1	As above

ML Scoring	1	Longer running, more exposed to capacity contention
Refresh Semantic Model	2	The Power BI REST API returns transient 429s under load

3. Data model

A star schema with four facts at three different grains, sharing conformed dimensions. Mixing grains in a single fact table is the most common way a voucher model breaks, so each business process gets its own fact.

What the drop actually supplies

The folder contains six files: four CSVs of data, a README, and a DataDictionary. The README's suggested model names FIVE tables against those four CSVs — the fifth, DimDate, has no source file at all. Its entry reads 'candidate should create a proper date table'. Every table in the delivered model is therefore tagged with one of two distinct origins where the README is concerned: supplied as a CSV, or named by the README and built here. Collapsing those into a single 'from the README' label would imply a DimDate.csv that has never existed.

Supplied file	Bytes	Becomes
MerchantReference.csv	2,143	DimMerchant
MerchantSales.csv	2,058,004	FactMerchantSales
VoucherRedemptions.csv	9,050,258	FactVoucherRedemptions
SupportTickets.csv	127,372	FactSupportTickets
README_BI_Interview_Data.txt	851	Requirements — names DimDate with no file
DataDictionary.csv	740	Reconciled in the column lineage below

Source column lineage, and what the supplied dictionary covers

The DataDictionary supplied with the drop describes 12 columns. The four CSVs contain 34. Two thirds of the model therefore rests on meanings inferred by profiling rather than stated by the data owner — that is an assumption, and it is recorded as one. Every source column is traced to the gold table it lands in, or marked as deliberately dropped with the reason. The build fails if a column is neither: a field that silently disappears between source and star schema is indistinguishable from a defect.

Measure	Count
Source columns across the four CSVs	34
Documented in the supplied DataDictionary	12 (35%)
Meaning inferred by profiling	22
Carried into the star schema	28
Deliberately dropped, reason recorded	6

Source	Column	Documented	Lands in	Note
MerchantReference	MerchantID	Yes	DimMerchant.MerchantKey	Surrogate key issued; natural key retained as MerchantID
MerchantReference	Merchant	Yes	DimMerchant.Merchant	
MerchantReference	Region	Yes	DimMerchant.Region	Single source of truth for region — the copy on the sales fact is dropped

MerchantReference	Channel	Yes	DimMerchant.Channel	
MerchantReference	ActiveStatus	Inferred	DimMerchant.ActiveStatus	Also surfaced as active_status on MerchantValueRisk — a churned merchant must not be scored as an attrition risk
MerchantReference	OnboardedDate	Inferred	DimMerchant.OnboardedDate	Derives TenureMonths and TenureBand
MerchantReference	AccountManager	Inferred	DimMerchant.AccountManager	The owner an at-risk merchant is routed to — carried onto MerchantValueRisk so the risk register is actionable
MerchantReference	BaseMonthlySalesTarget	Inferred	DimMerchant.BaseMonthlySalesTarget	Denominator for target attainment, which is the measure behind the tickets-vs-attainment question
MerchantSales	Date	Yes	FactMerchantSales.DateKey	Converted to an integer date key joining DimDate
MerchantSales	MerchantID	Inferred	FactMerchantSales.MerchantKey	Resolved to the surrogate key
MerchantSales	Merchant	Inferred	DROPPED	100% agreement with MerchantReference (profiled). Keeping it would allow two competing versions of merchant name in one model
MerchantSales	Region	Inferred	DROPPED	100% agreement with MerchantReference (profiled). Region belongs on the dimension
MerchantSales	Channel	Inferred	DROPPED	100% agreement with MerchantReference (profiled)
MerchantSales	VoucherType	Inferred	FactMerchantSales.VoucherTypeKey	Resolved against DimVoucherType — conformed with the redemption fact
MerchantSales	SalesValue	Yes	FactMerchantSales.SalesValue	Additive measure
MerchantSales	Transactions	Yes	FactMerchantSales.Transactions	Additive measure
VoucherRedemptions	VoucherID	Inferred	FactVoucherRedemptions.VoucherID	Degenerate dimension — unique by construction, which is why duplicate-redemption fraud cannot be detected in this extract
VoucherRedemptions	MerchantID	Inferred	FactVoucherRedemptions.MerchantKey	
VoucherRedemptions	Merchant	Inferred	DROPPED	Redundant copy of the merchant name — resolved via DimMerchant
VoucherRedemptions	SoldDate	Yes	FactVoucherRedemptions.SoldDateKey	ACTIVE relationship to DimDate
VoucherRedemptions	VoucherType	Inferred	FactVoucherRedemptions.VoucherTypeKey	
VoucherRedemptions	VoucherValue	Inferred	FactVoucherRedemptions.VoucherValue	
VoucherRedemptions	Redeemed	Yes	FactVoucherRedemptions.IsRedeemed	Yes/No text converted to 1/0 so it sums
VoucherRedemptions	RedeemedDate	Yes	FactVoucherRedemptions.RedeemedDateKey	INACTIVE relationship, activated by USERELATIONSHIP. Blank when unredeemed — kept blank, not defaulted
SupportTickets	TicketID	Inferred	FactSupportTickets.TicketID	Degenerate dimension
SupportTickets	Date	Inferred	FactSupportTickets.DateKey	Ticket created date
SupportTickets	MerchantID	Inferred	FactSupportTickets.MerchantKey	
SupportTickets	Merchant	Inferred	DROPPED	Redundant copy of the merchant name — resolved via DimMerchant
SupportTickets	Region	Inferred	DROPPED	Redundant copy of region — resolved via DimMerchant so one hierarchy governs every fact

SupportTickets	TicketType	Inferred	FactSupportTickets.TicketTypeKey	Resolved against DimTicketType
SupportTickets	Priority	Inferred	FactSupportTickets.PriorityKey	Resolved against DimPriority
SupportTickets	ResolutionHours	Yes	FactSupportTickets.ResolutionHours	
SupportTickets	SLAHours	Yes	FactSupportTickets.SLAHours	Stored ON the fact, not resolved from DimPriority, so a future SLA policy change cannot retrospectively restate whether a historic ticket breached
SupportTickets	Status	Inferred	FactSupportTickets.Status	Derives IsOpen. Open tickets have elapsed-hours in ResolutionHours rather than a final duration, so they are excluded from average resolution time

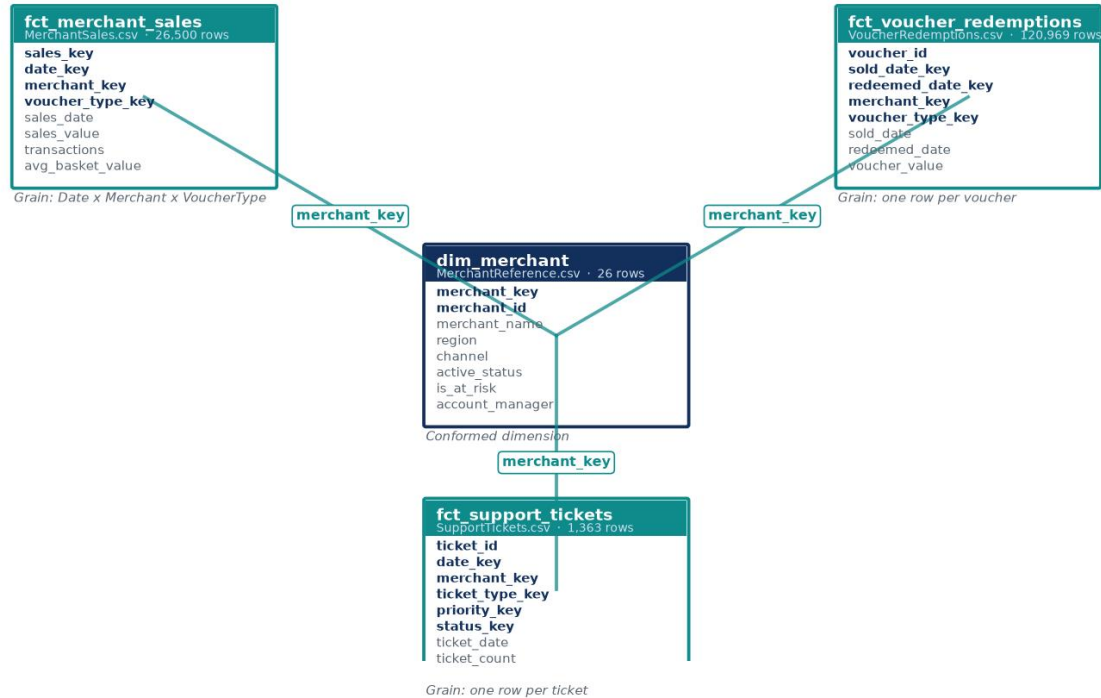
The four supplied tables on their own

Before the full model, the same structure reduced to only the tables that arrive as a CSV. Fourteen boxes is the correct answer to the brief but the wrong diagram for explaining the SHAPE of the model; four makes it obvious. One conformed dimension sits at the centre and three facts at three different grains each join to it on merchant_key. No fact joins to another fact — that is what prevents a fan trap and the silently multiplied totals it produces.

Every line in this diagram is read from the dbt manifest, specifically from the relationships tests, so the diagram cannot assert a join the project does not actually enforce. The seven further relationships to dimensions outside these four (dim_date, dim_voucher_type, dim_priority, dim_ticket_type, dim_ticket_status) are named beneath it rather than hidden — omitting date_key would misrepresent the model.

The four supplied tables — one conformed dimension, three facts

Every relationship shown is an enforced dbt relationships test.



Why fourteen tables when the README suggests five

The supplied README suggests a 5-table model. This solution has 14. Five come straight from the README; four exist because the brief's own report requirements (voucher type performance, ticket priority, SLA risk, backlog) need somewhere to put those attributes; one is a grain necessity; and four are extensions I added on judgement and have labelled as such. Presenting all 14 as though the brief demanded them would be overselling it — the honest split is a 10-table core plus 4 extensions.

Tier	Count	Tables
Supplied CSV	4	dim_merchant, fct_merchant_sales, fct_support_tickets, fct_voucher_redemptions
README-required	1	dim_date
Brief deliverable	4	dim_priority, dim_ticket_status, dim_ticket_type, dim_voucher_type
Grain necessity	1	fct_merchant_target
My extension	4	dim_merchant_history, mart_merchant_change_alerts, mart_merchant_scorecard, snap_merchant

Every table is justified individually below. The tier is carried through the dbt model descriptions, the ERD and the Excel data dictionary, so the same answer appears wherever the question is asked.

Table	Tier	Rows	Why it exists
dim_merchant	Supplied CSV	26	README: DimMerchant from MerchantReference.csv. Conformed dimension — the single filter path for every fact. Carries an Unknown (-1) member so a future unmatched fact row lands somewhere visible instead of vanishing from every total.
fct_merchant_sales	Supplied CSV	26,500	README: FactMerchantSales from MerchantSales.csv. Grain Date x Merchant x VoucherType. Merchant/Region/Channel were DROPPED from the fact — profiling proved 100% agreement with MerchantReference, so keeping both would allow two competing versions of 'Region' in one model.
fct_support_tickets	Supplied CSV	1,363	README: FactSupportTickets from SupportTickets.csv. One row per ticket. sla_hours is stored ON the fact, not resolved from the priority dimension, so a future SLA policy change cannot retrospectively restate whether a historic ticket breached.
fct_voucher_redemptions	Supplied CSV	120,969	README: FactVoucherRedemptions from VoucherRedemptions.csv. Accumulating snapshot — one row per voucher, updated when the second event (redemption) occurs. Carries TWO date foreign keys: sold_date_key active, redeemed_date_key inactive. Without that split, late redemptions are attributed back to the month of sale and a backlog is invisible.
dim_date	README-required	365	README: 'DimDate: candidate should create a proper date table'. This is the one table in the suggested model with NO source CSV — it is built, not loaded. Generated to full calendar-year boundaries rather than derived from the facts. Power BI time intelligence (DATEADD, SAMEPERIODLASTYEAR, TOTALYTD) does not error on a gapped calendar — it silently returns a wrong number. Contiguity is therefore tested, not assumed.
dim_priority	Brief deliverable	4	Brief section 5 requires 'ticket priority' and 'SLA risk'; section 4 requires an SLA breach indicator. Carries target_sla_hours plus observed median/P90 resolution time, and derives sla_is_achievable from them. This is what turns the headline operational finding into DATA rather

			than prose: Critical has a 12h target against a 74h P90, and the model says so in a sortable column.
dim_ticket_status	Brief deliverable	4	Brief section 5 requires ticket status and backlog visibility. Carries an explicit 'ownership' attribute that splits the backlog into 'awaiting us' (146 tickets) and 'awaiting customer' (93). A single '239 open' figure conflates a capacity problem with a chase-the-customer problem — different remediation entirely.
dim_ticket_type	Brief deliverable	6	Brief section 5 requires ticket analysis on the Operational page. Adds category (Fulfilment / Financial / Commercial) and impact area (Customer vs Merchant impacting) — groupings that do not exist in the source but are how operations actually triage.
dim_voucher_type	Brief deliverable	5	Brief section 5 requires 'voucher type performance' on the Merchant page. Used by BOTH the sales fact and the redemption fact — the textbook definition of a conformed dimension. Also carries category and margin band, which do not exist in the source and are maintained as a dbt seed so the commercial team can change them without a code deployment.
fct_merchant_target	Grain necessity	175	Brief section 4 requires period comparison and a target measure; BaseMonthlySalesTarget is supplied on MerchantReference. Could have been an attribute on dim_merchant — but it is a MONTHLY measure while sales are DAILY. On the dimension it would re-count once per fact row. A separate fact at month grain, joined only through shared dimensions, is the standard Kimball answer to multi-grain reporting. 25 merchants x 7 months = 175 rows. Targets are pro-rated by days covered so a part-month never shows a false shortfall.
dim_merchant_history	My extension	25	NOT requested. The Type 2 face of snap_merchant. dim_merchant answers 'what is this merchant TODAY' and is what the semantic model joins to. This answers 'what was this merchant WHEN that fact happened'. Loading only Type 2 and forcing every query through a date-range join would be purer and practically worse; loading only Type 1 loses history permanently. Both, clearly labelled, is the workable answer.
mart_merchant_change_alerts	My extension	0	NOT requested. Makes the snapshot actionable. Differs consecutive snapshot versions and emits one row per changed field with a severity. A merchant moving to 'At Risk' is Critical; a re-based sales target is High, because it silently invalidates every period-over-period attainment comparison unless someone is told. Capturing history that nobody reads is not worth the storage — this is what makes the snapshot earn its place. Currently 0 rows: no change has occurred since first capture.
mart_merchant_scorecard	My extension	25	NOT requested. Drill-through target for the Merchant page and feature table for the ML layer. Holds the composite Health Score. Computed in SQL rather than DAX deliberately: it needs percentile ranking across the whole merchant population (expensive and awkward in DAX) and it must be byte-identical in Power BI, the Excel pack and the ML feature set. One definition in the warehouse is what keeps three artefacts agreeing.
snap_merchant	My extension	25	NOT requested. Added because MerchantReference is a current-state extract. Each load overwrites the last, so active_status, account_manager, region and target history is destroyed at source. Two merchants are currently flagged 'At Risk' — 'when did that change, and did performance decline before or after?' is otherwise unanswerable forever. Uses the 'check' strategy because the source has no reliable last-modified column (OnboardedDate is when the merchant joined, not when the row was edited).

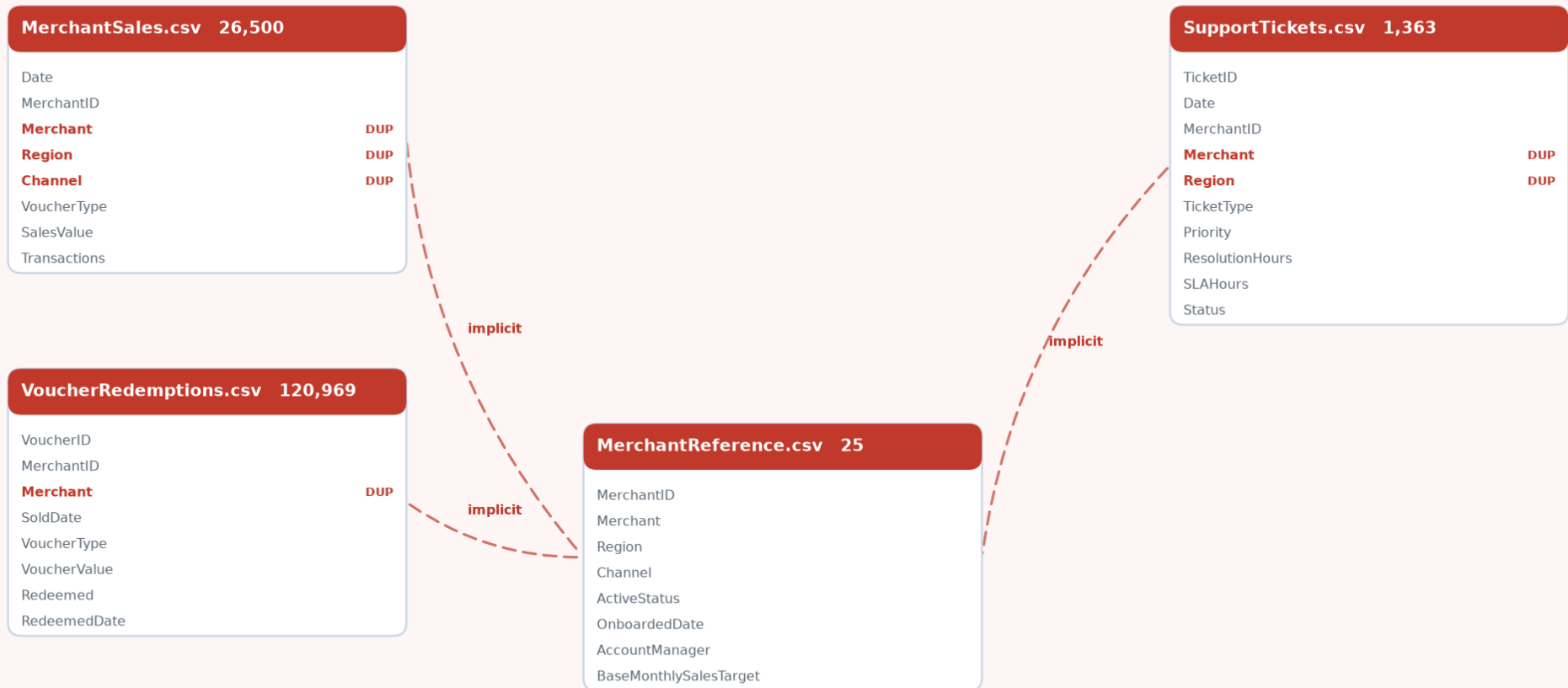
The fair criticism, and the answer

The fair criticism is that four dimensions of 4-6 rows each is over-normalisation, and some modellers would keep those as degenerate columns on the ticket fact. That was in fact the first design here — all three ticket dimensions unioned into one table behind a discriminator — and it was worse for two concrete reasons. The three foreign keys on `fct_support_tickets` could not be relationship-tested, because a test cannot distinguish 'priority_key resolves to a priority' from 'resolves to something, somewhere'. And Power BI cannot build three independent filter paths off one physical table without role-playing copies. Splitting them added 9 enforced foreign keys that previously could not exist.

Before and after

The four supplied files carry 6 duplicated attribute columns, no declared keys, no calendar, and every value as text. Every join between them is an implicit string match that nothing validates. The gold schema replaces that with conformed dimensions, typed columns and 15 foreign keys that are enforced by a test on every build — drop a join and the build fails.

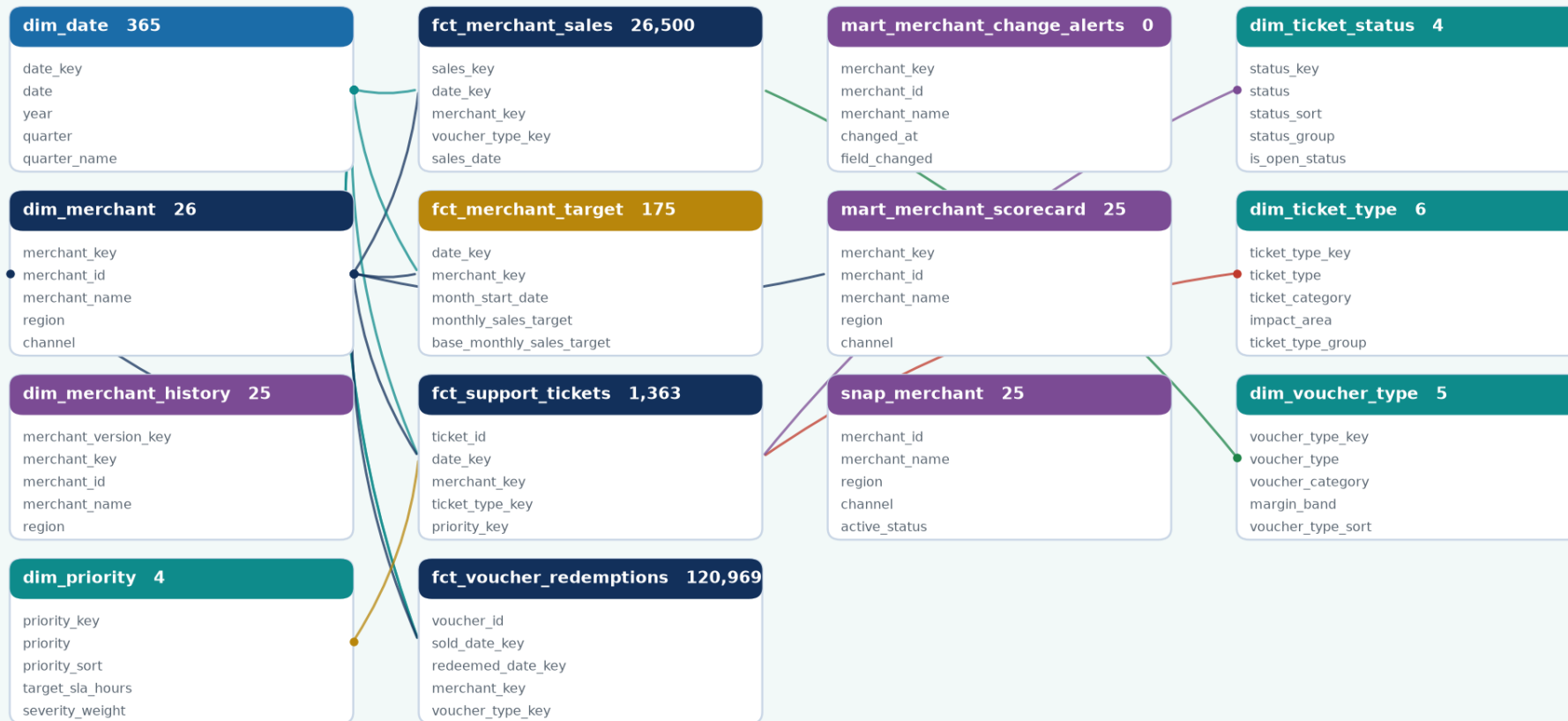
BEFORE — four flat files · 6 duplicated attribute columns · 0 enforced keys · every value text



Dashed arrows are implicit string joins. Nothing validates them, and nothing would notice if one stopped resolving.

BEFORE — the four files as delivered. Red dashed arrows are implicit string joins that nothing validates; DUP marks a duplicated attribute column.

AFTER — 14 tables · 15 ENFORCED foreign keys · conformed dimensions · contiguous calendar



■ Supplied CSV (4)

■ README-required (1)

■ Brief deliverable (4)

■ Grain necessity (1)

■ My extension (4)

How the tables join — every one an enforced dbt relationships test

- fct_merchant_sales.date_key → dim_date
- fct_voucher_redemptions.sold_date_key → dim_date
- fct_merchant_sales.merchant_key → dim_merchant
- fct_voucher_redemptions.merchant_key → dim_merchant
- fct_support_tickets.status_key → dim_ticket_status
- fct_merchant_target.date_key → dim_date
- fct_voucher_redemptions.redeemed_date_key → dim_date
- fct_merchant_target.merchant_key → dim_merchant
- mart_merchant_scorecard.merchant_key → dim_merchant
- fct_support_tickets.ticket_type_key → dim_ticket_type
- fct_support_tickets.date_key → dim_date
- dim_merchant_history.merchant_key → dim_merchant
- fct_support_tickets.merchant_key → dim_merchant
- fct_support_tickets.priority_key → dim_priority
- fct_merchant_sales.voucher_type_key → dim_voucher_type

AFTER — the gold star schema. Every arrow is an enforced relationships test, and each table is coloured by why it exists.

Delivered	Modelled	Why it matters
No date table	dim_date — 365 contiguous days, gap-tested	Time intelligence returns wrong answers, not errors, on a gapped calendar
Merchant/Region/Channel on 3 facts	Dropped; live only on dim_merchant	Profiling proved 100% agreement, so removal is lossless — and stops two versions of 'Region' existing
String joins, nothing enforced	15 relationships tests on every build	A broken join fails the pipeline instead of silently returning fewer rows
Everything text	Typed at silver	No implicit coercion hiding inside an aggregate
Repeated category strings	4 conformed dimensions	Sort order, grouping and metadata that do not exist in the source
Redemption date as a plain column	Role-playing date, ACTIVE + INACTIVE	Otherwise late redemptions attribute back to the sale month and a backlog is invisible
No history	snap_merchant SCD2 + dim_merchant_history	The source overwrites; 'when did this merchant become At Risk' was unanswerable
Two grains conflated	Separate facts + a recorded 4.2:1 ratio	Stops someone reporting a R43.5m 'break' that is not a break

Tables

Table	Type	Grain	Rows
dim_date	Dimension	One row per calendar day, contiguous, full year boundaries	365
dim_merchant	Dimension	One row per merchant + Unknown member	26
dim_merchant_history	Dimension (Type 2)	One row per merchant per version, with valid_from / valid_to	25
dim_voucher_type	Dimension	One row per voucher type	5
dim_priority	Dimension	One row per ticket priority	4
dim_ticket_type	Dimension	One row per ticket type	6
dim_ticket_status	Dimension	One row per ticket status	4
fct_merchant_sales	Fact (transaction)	Date × Merchant × VoucherType	26,500
fct_voucher_redemptions	Fact (accumulating snapshot)	One row per voucher	120,969
fct_support_tickets	Fact (transaction)	One row per ticket	1,363
fct_merchant_target	Fact (periodic snapshot)	Month × Merchant	175
mart_merchant_scorecard	Analytics mart	One row per merchant	25

Entity relationships

Generated from the dbt manifest, not drawn. dbt docs shows the DAG — which model BUILDS from which — and that is build lineage, not entity relationships: it cannot show that fct_support_tickets.priority_key joins to dim_priority, because the fact is built from staging, not from the dimension. The real foreign keys live in the schema tests. Every relationships test is an enforced statement that a column must resolve to another column, so this diagram cannot drift — drop a join and the test disappears with it.

From	Column	To
fct_merchant_sales	date_key	dim_date
fct_merchant_sales	merchant_key	dim_merchant

fct_merchant_sales	voucher_type_key	dim_voucher_type
fct_voucher_redemptions	sold_date_key (ACTIVE)	dim_date
fct_voucher_redemptions	redeemed_date_key (INACTIVE)	dim_date
fct_voucher_redemptions	merchant_key	dim_merchant
fct_support_tickets	date_key	dim_date
fct_support_tickets	merchant_key	dim_merchant
fct_support_tickets	ticket_type_key	dim_ticket_type
fct_support_tickets	priority_key	dim_priority
fct_support_tickets	status_key	dim_ticket_status
fct_merchant_target	date_key	dim_date
fct_merchant_target	merchant_key	dim_merchant
dim_merchant_history	merchant_key	dim_merchant
mart_merchant_scorecard	merchant_key	dim_merchant

15 enforced foreign keys. Note fct_voucher_redemptions carries TWO into dim_date — the role-playing date that makes a redemption backlog visible.

Normalisation, keys and the Kimball position

The model follows Kimball dimensional design (Kimball & Ross, The Data Warehouse Toolkit, 3rd edition). The choices below are the ones a reviewer is most likely to question, so each is stated with its reason rather than left implicit.

Normalised where it removes contradiction, denormalised where it removes joins

The source is denormalised without benefit: Merchant, Region and Channel appear on three fact files AND on the reference file — four places one value can disagree with itself, with no query made faster in exchange. Those nine columns were dropped from the facts after profiling proved 100 percent agreement, so removal was lossless. Region and Channel now live only on dim_merchant.

The dimensions themselves are deliberately NOT normalised further. Splitting Region, Channel and AccountManager into their own lookup tables is snowflaking, and Kimball is explicit that it should be resisted: dimension tables are geometrically smaller than fact tables, so normalising them has virtually no effect on database size, and the trade of dimension space for simplicity is almost always worth making. At 25 merchants the storage saving is nil while the cost — an extra join on every regional slice, and a longer filter-propagation chain in Power BI — is real.

Layer	Form	Why
Source CSVs	Denormalised, redundantly	Attributes repeated across four files with nothing keeping them in step
Facts	Normalised to keys and measures	One version of each attribute; a fact carries foreign keys, measures and degenerate dimensions only
Dimensions	Denormalised (flat), not snowflaked	One join to reach any attribute; storage saving from snowflaking is negligible at this size

The keys, and what each is for

Key type	In this model	Purpose
Primary key (dimension)	MerchantKey, DateKey, VoucherTypeKey, PriorityKey,	One row per key. The basis for referential integrity with every fact that joins to it.

	TicketTypeKey	
Surrogate key	Issued for each dimension; the natural key is retained alongside it	Insulates the warehouse from source key changes and lets a dimension hold several versions of the same natural key over time.
Natural key	MerchantID (M001), retained on dim_merchant	The business identifier. Kept so a merchant can still be traced back to the source system, and so a Type 2 dimension can group its versions.
Foreign key (fact)	15 enforced relationships, each with a dbt relationships test	The join path. A broken foreign key returns FEWER ROWS rather than an error, so these are tested on every build instead of trusted.
Role-playing key	SoldDateKey (active) and RedeemedDateKey (inactive), both to DimDate	One dimension playing two roles. Sold date is the default because every voucher has one; redeemed date is null until redemption, so activating it would silently drop unredeemed vouchers from any date-filtered total.
Degenerate dimension	VoucherID and TicketID, held on the fact with no dimension table	An identifier with no attributes of its own. Kimball's guidance is to leave it on the fact rather than build a dimension that would hold nothing but the key.
Composite / compound key	fct_merchant_sales is unique on date x merchant x voucher type	The declared grain. Tested with a composite uniqueness test — the check that a fact has not silently gained a second row per key.
Unknown member	MerchantKey = -1 on dim_merchant	An unmatched fact row lands on a visible Unknown member instead of vanishing from every total. This is why dim_merchant has 26 rows against 25 merchants.

What else in Kimball this build applies

Principle	Applied here
Declare the grain before anything else	Each fact states its grain explicitly and is tested against it: sales at date x merchant x voucher type, redemptions one row per voucher, tickets one row per ticket.
Never mix grains in one fact table	Monthly targets are a separate 175-row fact, not a column on dim_merchant. On the dimension a monthly value would re-count once per daily fact row — the fan trap — and inflate every attainment figure.
Conformed dimensions	dim_date, dim_merchant and dim_voucher_type are shared across facts, so measures from different business processes can be compared side by side.
Facts join only through dimensions	No fact joins directly to another fact. Q7 in the SQL pack is written specifically to show the target and sales facts meeting only via shared dimension keys.
Slowly changing dimensions	MerchantReference is a current-state extract. A Type 2 snapshot preserves status, owner and target history that the source overwrites on every load.
Accumulating snapshot	The voucher fact is one row per voucher, updated when the second event (redemption) occurs — which is why it carries two date keys rather than two rows.
Additive, semi-additive, non-additive	SalesValue and Transactions are fully additive. Rates and percentages are held as measures rather than stored columns, so they are never summed by accident.

Modelling decisions worth defending

Role-playing dates on the redemption fact. A voucher has two events — sold and redeemed — so the fact carries both sold_date_key and redeemed_date_key. In the semantic model, dim_date joins sold_date_key on the ACTIVE relationship and redeemed_date_key on an INACTIVE one, activated inside specific measures via USERELATIONSHIP. This is the single most common place a voucher model goes wrong: reporting redemptions

on the sale date attributes late redemptions back to the month of issue, which makes a redemption backlog completely invisible.

Descriptive attributes dropped from the facts. Merchant, Region and Channel appear on both the sales fact and MerchantReference. Profiling confirmed 100% agreement across every row, so they were dropped from the facts — losslessly. Keeping them would allow two competing versions of 'Region' to exist in the model, which is how a report ends up with two different regional totals on two different pages.

SLA hours stored on the fact, not resolved from the dimension. They agree today. But a future SLA policy change must not retrospectively restate whether a historic ticket breached — a ticket is judged against the SLA in force when it was raised. `dim_priority.target_sla_hours` therefore describes CURRENT policy, and the fact records what was actually promised at the time.

Targets in a separate fact at month grain. `fct_merchant_target` is a periodic snapshot at a coarser grain than the daily sales fact. Forcing a monthly target into a daily fact would break every additive measure. The two facts join only through shared dimensions, which is the standard Kimball answer to multi-grain reporting. Targets are pro-rated by days covered so a part-month never shows a false shortfall.

An Unknown (-1) dimension member. Emitted deliberately so a future fact row with an unmatched merchant lands somewhere visible in the report rather than silently disappearing from every total.

Both a Type 1 and a Type 2 merchant dimension. `dim_merchant` is current state and is what the semantic model joins to — 'show me sales for merchants CURRENTLY in Gauteng' is what a regional manager asking about their own patch actually means. `dim_merchant_history` is versioned and answers point-in-time questions: 'what were sales for merchants that were in Gauteng AT THE TIME'. Loading only the Type 2 version and forcing every query through a date-range join would be technically purer and practically worse; loading only Type 1 loses history permanently. Both, clearly labelled, is the workable answer.

Diagnostics carried in the dimension, not just the report. `dim_priority` stores the observed median and 90th-percentile resolution time next to the target SLA, and derives `sla_is_achievable` from them. The SLA finding is therefore data rather than prose: Critical carries a 12-hour target against a 74-hour 90th percentile, and the model says so directly. A reader can sort on it; they cannot sort on a paragraph.

4. Transformation layer

Business rules, defined once

Every threshold lives in a single place (dbt_project.yml vars) and is inherited by the SQL models, the PySpark notebooks and the DAX measure library. A change propagates consistently instead of being applied in three places and forgotten in a fourth.

Rule	Value	Basis
Delayed redemption	> 7 days	No threshold supplied. 7 days sits above the 75th percentile (5 days), so it flags a genuine tail rather than routine behaviour
Voucher expiry / breakage	90 days	No expiry rule supplied; 90 days is a common industry default. Affects breakage only
High priority	High, Critical	Used for the SLA concentration analysis
Redemption integrity	Flag AND valid non-retrograde date must agree	Trusting the flag alone would let a corrupt feed inflate the most scrutinised KPI in the report

SQL report pack — the queries behind the numbers

Eight queries answer the brief directly from the gold star schema, in docs/report_queries.sql. Each joins through the conformed dimensions rather than reading a fact in isolation: the join path IS the argument for the model. Each was executed against the warehouse and the results captured as rendered output in docs/screenshots/sql, so the image and the data cannot drift apart.

Q1 Top merchants by sales value AND transaction volume

The brief asks for both. They are reported together because the rankings can disagree — a merchant with a larger basket leads on value while trailing on volume.

```
SELECT
  m.merchant_name,
  m.region,
  m.channel,
  SUM(s.sales_value) AS total_sales,
  SUM(s.transactions) AS total_transactions,
  SUM(s.sales_value) / NULLIF(SUM(s.transactions), 0) AS avg_basket,
  RANK() OVER (ORDER BY SUM(s.sales_value) DESC) AS rank_by_value,
  RANK() OVER (ORDER BY SUM(s.transactions) DESC) AS rank_by_volume
FROM
  main_marts.fct_merchant_sales s
INNER JOIN
  main_marts.dim_merchant m ON m.merchant_key = s.merchant_key
INNER JOIN
  main_marts.dim_date d ON d.date_key = s.date_key
GROUP BY
  m.merchant_name, m.region, m.channel
ORDER BY
  total_sales DESC
LIMIT
  10
```

RESULT — 10 row(s)

merchant_name	region	channel	total_sales	total_transactions	avg_basket	rank_by_value	rank_by_volume
Durban Cash Hub	Western Cape	Retail	5776118.74	45371.0	127.31	1	1
Kudu Digital Kiosk	KwaZulu-Natal	Agent Network	4535329.73	35458.0	127.91	2	3
Zebra Route Services	Gauteng	Agent Network	4519761.94	35910.0	125.86	3	2
Karoo Quick Shop	Gauteng	Wholesale	4395067.11	34657.0	126.82	4	4
Lighthouse General Dealer	Gauteng	Wholesale	3965086.39	31124.0	127.40	5	5
North Coast Connect	Free State	Online	3731381.85	29341.0	127.17	6	6
Ubuntu Trading Post	KwaZulu-Natal	Retail	3548946.51	27865.0	127.36	7	7
Table Bay Express	Eastern Cape	Wholesale	3325306.26	26856.0	127.62	8	8
Blue Crane Trading	Gauteng	Retail	3265565.26	25532.0	127.90	9	9
Liberty Lane Stores	Western Cape	Wholesale	3185149.79	24828.0	128.29	10	10

Q2 Redemption rate by voucher type

Joined through dim_voucher_type so the type attributes come from the conformed dimension, not from a string repeated on the fact.

```
SELECT
  vt.voucher_type,
  COUNT(*) AS vouchers_sold,
  SUM(CASE WHEN r.is_redeemed THEN 1 ELSE 0 END) AS vouchers_redeemed,
  100.0 * SUM(CASE WHEN r.is_redeemed THEN 1 ELSE 0 END) / COUNT(*) AS redemption_rate_pct,
  AVG(r.days_to_redeem) AS avg_days_to_redeem,
  SUM(CASE WHEN NOT r.is_redeemed THEN r.voucher_value ELSE 0 END) AS outstanding_liability
FROM
  main_marts.fct_voucher_redemptions r
INNER JOIN
  main_marts.dim_voucher_type vt ON vt.voucher_type_key = r.voucher_type_key
GROUP BY
  vt.voucher_type
ORDER BY
  redemption_rate_pct DESC
```

RESULT — 5 row(s)

voucher_type	vouchers_sold	vouchers_redeemed	redemption_rate_pct	avg_days_to_redeem	outstanding_liability
Airtime	26253	24370.0	92.83	3.55	340978.83
Electricity	24691	21677.0	87.79	3.56	557878.98
Groceries	24856	20549.0	82.67	3.57	784888.34
Bill Payment	22489	18922.0	80.14	3.70	824709.65
Gaming	22680	17225.0	75.95	3.56	1033107.28

Q3 Regional trend — which region is declining

A region is only 'declining' relative to its own peak, so the query computes each region's peak month and the gap to it, rather than ranking on the latest month alone.

```
WITH monthly AS (
  SELECT
    m.region,
    d.year_month,
    SUM(s.sales_value) AS sales
  FROM
    main_marts.fct_merchant_sales s
  INNER JOIN
    main_marts.dim_merchant m ON m.merchant_key = s.merchant_key
  INNER JOIN
    main_marts.dim_date d ON d.date_key = s.date_key
  GROUP BY
    m.region, d.year_month
)
SELECT
  region,
  MAX(sales) AS peak_month_sales,
  MAX_BY(year_month, sales) AS peak_month,
  MAX_BY(sales, year_month) AS latest_month_sales,
  MAX_BY(year_month, year_month) AS latest_month,
  100.0 * (MAX_BY(sales, year_month) - MAX(sales))
  / NULLIF(MAX(sales), 0) AS pct_below_own_peak
FROM
  monthly
GROUP BY
  region
ORDER BY
  pct_below_own_peak
```

RESULT — 5 row(s)

region	peak_month_sales	peak_month	latest_month_sales	latest_month	pct_below_own_peak
Eastern Cape	1400867.94	2026-05	1264270.55	2026-07	-9.75
Free State	2836918.29	2026-07	2836918.29	2026-07	0.00
KwaZulu-Natal	1680558.12	2026-07	1680558.12	2026-07	0.00
Gauteng	2597861.85	2026-07	2597861.85	2026-07	0.00
Western Cape	2087340.68	2026-07	2087340.68	2026-07	0.00

Q4 Support tickets vs sales performance

The naive version correlates ticket count with sales and concludes that tickets cause decline. This joins tickets to the MONTHLY sales panel so volume and performance are compared at the same grain, and reports both together for judgement.

```
WITH sales_m AS (
  SELECT
    s.merchant_key, d.year_month, SUM(s.sales_value) AS sales,
    SUM(t.transactions) AS txns
  FROM
    main_marts.fct_merchant_sales s
  INNER JOIN
    main_marts.dim_date d ON d.date_key = s.date_key
  GROUP BY
    s.merchant_key, d.year_month
),
tix_m AS (
  SELECT
    t.merchant_key, d.year_month, COUNT(*) AS tickets,
    SUM(CASE WHEN t.is_sla_breach THEN 1 ELSE 0 END) AS breaches
  FROM
    main_marts.fct_support_tickets t
  INNER JOIN
    main_marts.dim_date d ON d.date_key = t.date_key
  GROUP BY
    t.merchant_key, d.year_month
)
SELECT
  m.merchant_name,
  SUM(s.sales) AS total_sales,
  SUM(COALESCE(x.tickets, 0)) AS tickets,
  SUM(COALESCE(x.breaches, 0)) AS sla_breaches,
  1000.0 * SUM(COALESCE(x.tickets, 0))
  / NULLIF(SUM(s.txns), 0) AS tickets_per_1k_txn
FROM
  sales_m s
INNER JOIN
  main_marts.dim_merchant m ON m.merchant_key = s.merchant_key
LEFT JOIN
  tix_m x ON x.merchant_key = s.merchant_key
  AND x.year_month = s.year_month
GROUP BY
  m.merchant_name
ORDER BY
  tickets_per_1k_txn DESC
LIMIT
  10
```

RESULT — 10 row(s)

merchant_name	total_sales	tickets	sla_breaches	tickets_per_1k_txn
Highveld Voucher Depot	505737.91	66.0	16.0	18.77
Umlanga Value Mart	684341.39	69.0	13.0	13.93
Berg River Retail	1077657.92	69.0	16.0	8.42
Cape Point Cellular	776438.09	44.0	15.0	7.76
Valley View Retail	822002.78	39.0	10.0	6.47
Mango Tree Mobile	2196476.67	109.0	32.0	6.44
Nelson Bay Traders	1694276.03	76.0	16.0	5.88
Marula Market	2138321.62	62.0	13.0	3.75
Jozi Corner Store	1502983.68	43.0	11.0	3.74
Sunbird Topups	1693657.85	43.0	7.0	3.29

Q5 Where management should focus — value at risk, not percentage decline

Ranking by percentage change sends the account team to the smallest merchants. This ranks by the RAND value at risk, joining the scorecard to the risk register.

```
SELECT
  v.merchant_name,
  v.region,
  v.account_manager,
  v.health_score,
  v.health_band,
  v.total_sales,
  v.revenue_at_risk_annualised,
  v.attrition_risk_band
FROM
  main_marts.mart_merchant_value_risk v
ORDER BY
  v.revenue_at_risk_annualised DESC
LIMIT
  10
```

RESULT — 10 row(s)

merchant_name	region	account_manager	health_score	health_band	total_sales	revenue_at_risk_annualised	attrition_risk_band
Umlanga Value Mart	Free State	A. Naidoo	17.3	Critical	684341.39	571517.84	High
Table Bay Express	Eastern Cape	A. Naidoo	24.0	Critical	3325306.26	354163.20	High
Pretoria PayPoint	Eastern Cape	L. Govender	31.3	Critical	2066202.17	154045.00	High
Mzansi Mini Market	Eastern Cape	R. Pillay	40.4	Watch	2613183.07	87168.92	Medium
Cape Point Cellular	Eastern Cape	R. Pillay	45.0	Watch	776438.09	48625.72	Medium
Kudu Digital Kiosk	KwaZulu-Natal	A. Naidoo	75.6	Star	4535329.73	0.00	Low
Karoo Quick Shop	Gauteng	S. Jacobs	62.9	Healthy	4395067.11	0.00	Low
Lighthouse General Dealer	Gauteng	T. Mokoena	62.1	Healthy	3965086.39	0.00	Low
North Coast Connect	Free State	A. Naidoo	34.8	Critical	3731381.85	0.00	Medium
Blue Crane Trading	Gauteng	S. Jacobs	62.1	Healthy	3265565.26	0.00	Low

Q6 SLA performance by priority — the operational finding

Joins the ticket fact to dim_priority so the SLA TARGET and the observed distribution sit side by side. This is what shows the queue running in reverse priority order.

```
SELECT      p.priority,
            p.target_sla_hours,
            COUNT(*)                               AS tickets,
            MEDIAN(t.resolution_hours)              AS median_hours,
            QUANTILE_CONT(t.resolution_hours, 0.90)  AS p90_hours,
            100.0 * SUM(CASE WHEN t.is_sla_breach THEN 1 ELSE 0 END)
            / COUNT(*)                             AS breach_rate_pct
FROM        main_marts.fct_support_tickets t
INNER JOIN  main_marts.dim_priority p ON p.priority_key = t.priority_key
GROUP BY   p.priority, p.priority_sort, p.target_sla_hours
ORDER BY   p.priority_sort
```

RESULT — 4 row(s)

priority	target_sla_hours	tickets	median_hours	p90_hours	breach_rate_pct
Critical	12	119	52.70	74.46	98.32
High	24	270	36.20	56.84	82.22
Medium	36	454	18.50	27.90	3.96
Low	48	520	10.45	16.11	0.19

Q7 Target attainment — the multi-grain join

Monthly targets against daily sales. The two facts are joined ONLY through shared dimension keys, never directly to each other — joining facts to facts is what produces a fan trap and silently multiplied totals.

```
SELECT      m.merchant_name,
            SUM(t.monthly_sales_target)             AS target,
            SUM(a.sales)                           AS actual,
            100.0 * SUM(a.sales) / NULLIF(SUM(t.monthly_sales_target), 0)
            AS attainment_pct
FROM        main_marts.fct_merchant_target t
INNER JOIN  main_marts.dim_merchant m ON m.merchant_key = t.merchant_key
LEFT JOIN (
    SELECT   s.merchant_key, d.year_month, SUM(s.sales_value) AS sales
    FROM     main_marts.fct_merchant_sales s
    INNER JOIN main_marts.dim_date d ON d.date_key = s.date_key
    GROUP BY s.merchant_key, d.year_month
) a ON a.merchant_key = t.merchant_key
    AND a.year_month = STRFTIME(t.month_start_date, 'YY-mm')
GROUP BY   m.merchant_name
ORDER BY   attainment_pct
LIMIT     10
```

RESULT — 10 row(s)

merchant_name	target	actual	attainment_pct
Nelson Bay Traders	445270.00	1694276.03	380.51
Marula Market	560203.00	2138321.62	381.70
Mango Tree Mobile	574959.00	2196476.67	382.02
Highveld Voucher Depot	131830.00	505737.91	383.61
Ushlanga Value Mart	154539.00	604341.39	442.83
Wild Coast Convenience	632512.65	3040622.49	480.72
Jozzi Corner Store	311213.00	1562983.68	482.94
North Coast Connect	705907.86	3731381.85	528.59
Berg River Retail	200809.00	1077657.92	536.66
Mzansi Mini Market	483203.00	2613183.07	540.80

Q8 Reconciliation — the two populations that must NOT agree

Sales transactions and voucher rows count different things. The variance is expected, and the query states the ratio so nobody escalates it as a break.

```
SELECT      'Sales fact' AS population,
            COUNT(*)      AS rows_in_fact,
            SUM(s.transactions) AS transactions,
            SUM(s.sales_value) AS value
FROM        main_marts.fct_merchant_sales s
UNION ALL
SELECT      'Voucher fact',
            COUNT(*),
            NULL,
            SUM(r.voucher_value)
FROM        main_marts.fct_voucher_redemptions r
```

RESULT — 2 row(s)

population	rows_in_fact	transactions	value
Sales fact	26500	510127.0	65521298.75
Voucher fact	120969	NaN	22019852.75

A finding about the supplied targets

Target attainment is above 100% for every merchant in the dataset — the range is 381% to 916%, and the portfolio total is 613.7% (R65,521,299 actual against R10,676,900 of target). This was checked before being reported: the join is correct and the figure reproduces from the raw CSV. BaseMonthlySalesTarget is simply not on the same scale as observed sales in this synthetic data — Blue Crane Trading carries a R60,133 monthly target against roughly R466,509 of actual monthly sales.

The consequence is worth stating plainly, because it changes how the measure may be used. Attainment here is a RELATIVE ranking device — the 2.41x spread between the weakest and strongest merchant is meaningful — but the absolute percentage is not, and no target-hit or target-miss commentary should be built on it. It also reinforces the Q4 conclusion: with a fixed denominator per merchant, attainment is close to a rescaling of sales, which is exactly why the apparent tickets-versus-attainment correlation collapses once size is controlled for.

What dbt is used for, and what it earned

dbt is not doing anything here that Python could not do. Naming what it actually contributed is more useful than claiming it was indispensable, so each item below is the concrete thing that would be lost without it.

What dbt provides	In this project	What is lost without it
Enforced referential integrity	15 relationships tests between facts and dimensions	The BEFORE model joins on a string nobody validates. A broken join returns FEWER ROWS rather than an error — the totals simply get quietly smaller and no one is told. These fail the build instead.
Grain and uniqueness contracts	22 unique, 56 not_null, 2 composite-key tests	Duplicate-proofing becomes a claim rather than a check. A fact silently gaining a second row per key double-counts every measure built on it.
Domain constraints	16 accepted_values, 11 expression tests, 5 range tests	A new Priority value or a negative sales figure enters the model and reaches the report as a legitimate-looking number.
Change history (SCD2)	1 snapshot over MerchantReference	This is the strongest single case. The source is a CURRENT-STATE extract: each load overwrites active_status, account_manager and target. Without the snapshot that history is destroyed at source and is unrecoverable — 'when did this merchant become At Risk, and did sales fall before or after?' can never be answered.
Business logic held outside code	5 seeds (voucher category, margin band, priority and status metadata)	The commercial team needs a code deployment to change a margin band.
Lineage and documentation from the build	21 models; the ERD, DAG and column lineage in this report are generated from manifest.json	Documentation is drawn by hand and starts drifting from the code the day it is written.
One codebase, two engines	Adapter-dispatch macros in macros/portability.sql	A local engine for development (seconds, no cloud cost) and Fabric for production would be two separately-maintained SQL dialects that gradually disagree.

The honest limit of the argument: the transformation itself could have been written in pandas, and in fact it was — the Python pipeline in scripts/02 and scripts/03 produces the same gold tables independently. That duplication is deliberate. Two implementations of the same business logic should agree, and the reconciliation in section 8 is what checks that they do. It is what caught the R984,046 date-window defect: the calendar SPAN was being used where the reporting WINDOW was meant, and only a disagreement between the two implementations exposed it.

dbt project

21 models across staging, intermediate and marts, plus 1 snapshot, 5 seeds and 132 tests. The project runs against a local in development (seconds, no cloud cost) and the Fabric Warehouse in production via dbt-fabric. Identical SQL, only the adapter changes.

Layer	Models	Purpose
staging	4 views	Type, trim, deduplicate at the declared grain, apply integrity rules
intermediate	2 ephemeral	int_merchant_monthly (merchant × month spine, zero-filled) and int_merchant_momentum (period comparisons). Inlined as CTEs — no extra storage, one definition
snapshots	1	snap_merchant — Type 2 capture of slowly-changing merchant attributes
marts / core	10	6 conformed dimensions + 4 facts, plus the Type 2 dimension

marts / analytics	1	mart_merchant_scorecard — drill-through target and ML features
-------------------	---	--

Two modelling changes were made after the first build, both to close real gaps:

Three ticket dimensions instead of one combined table. Ticket type, priority and status were originally unioned into a single table behind a dimension_type discriminator. That had two costs: the three foreign keys on fct_support_tickets could not be tested for referential integrity — a relationships test could not distinguish 'priority_key resolves to a priority' from 'resolves to something, somewhere' — and Power BI cannot build three independent filter paths off one physical table without role-playing copies. Splitting them added 9 referential tests that previously could not exist. Six rows is not too small to deserve its own dimension; join clarity matters more than table count.

A Type 2 snapshot on merchant attributes. MerchantReference is a current-state extract — each load overwrites the last, so active_status, account_manager and target history is destroyed at source. That matters: two merchants are currently flagged 'At Risk', and 'when did that change, and did performance decline before or after?' is unanswerable without history. The snapshot uses the check strategy rather than timestamp, because the source has no reliable last-modified column (OnboardedDate is when the merchant joined, not when the row was edited).

Test category	Count	Examples
Uniqueness / not-null	58	Primary keys on every dimension, fact and mart
Referential integrity	18	Every fact foreign key resolves to its own dimension
Accepted values	14	Region, Channel, Priority, Status, SLAHours, quality_flag
Range / expectation	16	sales_value ≥ 0; resolution_hours between 0 and 720
Row-level logic	10	is_sla_breach must equal (resolution_hours > sla_hours)
Completeness	6	The scorecard must cover every real merchant; every merchant must have 7 months observed
Singular (business)	4	dim_date contiguity; bronze→gold revenue reconciliation; redemption rate plausibility; no future-dated facts
Source freshness	3	Warn at 26h, error at 48h
SCD Type 2 behaviour	7	Separate harness: simulates a change, asserts history was captured, restores state

Validating the snapshot rather than declaring it

A snapshot that has only ever seen one version of each row is indistinguishable from one that is silently broken. scripts/_test_scd2.py simulates a real merchant change (Umhlanga Value Mart moving to 'At Risk' with a new account manager), re-runs the snapshot, and asserts seven properties: a new version row was created, exactly one version is current, the current version carries the new value, the superseded version retains the old value and was closed off, version numbering is contiguous, and unchanged merchants did NOT gain a version. All seven pass, and the harness restores the original state afterwards.

The highest-value test in the project

dim_date contiguity. Power BI time-intelligence functions (DATEADD, SAMEPERIODLASTYEAR, TOTALYTD) do not raise an error on a date table with a missing day — they silently return a wrong number. A single absent date quietly understates a month-on-month comparison and nobody notices until someone reconciles by hand. The test asserts that the row count equals the span between the minimum and maximum date, which cannot be true if there is a gap.

Data governance

Governance here means the questions a reviewer, an auditor or a downstream AI tool would ask of a number, and whether this model can answer them. Each row states the position taken and what backs it.

Question	Position in this build
Where did this number come from?	Every figure resolves to a source column through docs/source_column_lineage.csv, to the SQL that produced it through docs/report_queries.sql, and to the model that built it through the dbt manifest. The ERDs and lineage diagrams are generated from that manifest, so documentation cannot drift from code.
What does this field mean?	142 columns across 12 tables are defined in the data dictionary below. Each definition is attributed: 13 come from the supplied DataDictionary, 16 are traced through lineage, 93 are defined in this build and 20 follow a naming convention. None are left undefined.
Which values are facts and which are assumptions?	Of 34 source columns, 12 are documented by the data owner and 22 have meanings inferred by profiling. Every threshold that was not supplied — 7-day delayed redemption, 90-day breakage, R450/hour support cost, Health Score weights — is listed in the assumptions register rather than presented as given.
Can a consumer be given data without production access?	Yes, and that is the purpose of the gold layer. It is a curated, tested, documented set of tables in a separate Warehouse. A reporting tool, an analyst or an AI service connects there and never touches a production database.
How do we know the data is right?	Three independent layers: 132 dbt tests on structure and referential integrity, 28 reconciliation controls comparing two independent implementations, and 33 idempotency assertions. The build stops on failure rather than publishing.
Who owns a merchant when something is wrong?	account_manager is carried onto the risk register, so an at-risk merchant resolves to a named owner rather than to a queue.
What happens when reference data changes?	MerchantReference is a current-state extract — each load overwrites status, owner and target. A Type 2 snapshot captures the change, and mart_merchant_change_alerts raises status changes. Without it that history is destroyed at source and is unrecoverable.
Is there personal data here?	No. There is no customer identifier anywhere in the four source files — no name, account, PIN or device. This is why CLV, churn and duplicate-redemption fraud are modelled at MERCHANT level. Merchant names and account managers are business contact data, not consumer PII; a real deployment would still classify and label them.
Can a load be re-run safely?	Yes. Bronze writes are overwrite, not append — verified by running the ingestion twice with identical row counts — and the gold build is verified idempotent by 33 assertions.
What is NOT covered?	Row-level security, sensitivity labelling and retention policy are not implemented. They are deployment concerns needing an organisation's classification scheme, and are listed here rather than left silent.

Governance as an operating model

Governance is everyone's business, but every data asset needs one named owner. Shared responsibility for behaviour; single accountability for each asset. Both halves are needed — bad data almost always enters at the point of capture, which no amount of downstream engineering repairs, while an asset owned by "the business" is an asset nobody answers for when a number is wrong.

Role	Accountable for	In this solution
Data owner (business)	Whether the figure is correct, and who may see it	account_manager is carried onto the risk register so an at-risk merchant resolves to a person rather than to a queue.
Data steward	What each field means	142 columns defined, each attributed to its source: 13 from the supplied dictionary, 16 traced through lineage, 93 defined here, 20 by naming convention.
Data engineer	The controls that enforce the first	132 dbt tests, 28 reconciliation controls and 33 idempotency

	two	assertions, all of which stop the build rather than warn after publication.
Consumer	Raising a suspect number instead of working around it	Every figure is traceable to the SQL and the model that produced it, so a challenge can be resolved rather than debated.

The distinction worth drawing is between a policy and an operating model. A policy that is not enforced is documentation. Governance becomes real when it sits IN the pipeline: a failing test stops a build, a sensitivity label travels with lineage, and an uncertified model is visibly uncertified. That shifts the default from asking people to comply to making compliance the path of least resistance.

Platform choices — storage modes, fallback and mirroring

Three Fabric capabilities determine how a semantic model gets its data. They are set out here because the choice between them is an architecture decision rather than a setting, and because this solution deliberately uses one of them today and is built for another in production.

Mode	Where the data sits	Freshness	Trade-off
Import	Copied into the in-memory engine	Stale until the next refresh	Fastest queries; a refresh window and a second copy of the data.
DirectQuery	Stays in the source; each visual issues a query	Always current	No copy and no refresh, but every interaction pays a query cost.
Direct Lake	Delta/Parquet in OneLake, paged into memory on demand	Current, no refresh needed	Near-import speed with no copy — but requires Fabric capacity, Delta storage, and it falls back to DirectQuery under conditions listed below.

This submission ships an IMPORT model, because the deliverable is a .pbit template that must open on a reviewer's machine without a Fabric capacity attached. The gold layer is nonetheless Direct Lake ready: it is written as Delta in OneLake, and the semantic model carries no calculated columns or calculated tables, which are among the things that force a fallback.

Fallback — what it is, and what it is not

Fallback is frequently misunderstood as a standby copy of the data. It is not. There is no fallback table and nothing is duplicated. It is a QUERY-TIME behaviour: when Direct Lake cannot answer a query from the Parquet files, that single query is redirected to run as SQL against the SQL analytics endpoint. The data is identical; only the execution path changes, and the next query may well be served by Direct Lake again.

Common triggers: capacity guardrails exceeded (row limits scale with the SKU); a view rather than a table, since a view has no Delta files to page; unsupported data types; certain row-level security configurations at the SQL endpoint; and some calculated columns and tables.

Why fallback deserves attention

Fallback is SILENT. The report simply becomes slower and nothing reports why. The model property DirectLakeBehavior controls it: Automatic falls back when needed, DirectQueryOnly never uses Direct Lake, and DirectLakeOnly makes the query FAIL instead of falling back. Setting DirectLakeOnly during testing converts an invisible performance problem into a loud error that names the offending table.

Mirroring — where the data comes from

Mirroring continuously replicates an external database into OneLake as Delta tables, typically seconds to minutes behind the source. Supported sources include Azure SQL Database and Managed Instance, Cosmos DB, Snowflake, Databricks and PostgreSQL, with open mirroring available for others. The replica is read-only in Fabric, so it cannot write back to production, and mirrored storage is free up to a limit tied to the capacity.

The three capabilities compose into one path: mirroring lands production data in OneLake as Delta, Direct Lake reads it without a refresh, and Power BI reports on it near-live — without a pipeline to maintain and without any consumer touching the production database.

The honest limit: mirroring replaces the extract and load, not the modelling. It lands data in SOURCE shape — normalised operational tables, source naming, no conformed dimensions and no declared grain. A mirrored database is not a star schema. The transformation layer in this solution is exactly what would sit on top of it, with the mirrored tables declared as dbt sources in place of the bronze tables used here.

Data dictionary

142 columns across 12 tables. Generated from the gold tables at build time rather than maintained by hand, so it cannot fall out of step with the data. Every definition carries its source: 93 defined in this build, 20 convention, 16 lineage, 13 supplied datadictionary.

DimDate

Column	Type	Nulls	Definition	Source
Date	Text	0	Daily sales date	Supplied DataDictionary
DateKey	Integer	0	Surrogate key. Joins to the matching dimension.	Convention
Year	Integer	0	Calendar year.	Defined in this build
QuarterNumber	Integer	0	Quarter as 1-4, for sorting.	Defined in this build
Quarter	Text	0	Calendar quarter, e.g. Q3.	Defined in this build
MonthNumber	Integer	0	Month as 1-12, for sorting.	Defined in this build
MonthName	Text	0	Full month name.	Defined in this build
MonthShort	Text	0	Three-letter month abbreviation.	Defined in this build
MonthYear	Text	0	Month and year for display, e.g. Jul 2026.	Defined in this build
MonthYearSort	Integer	0	Integer YYYYMM so MonthYear sorts chronologically, not alphabetically.	Defined in this build
Day	Integer	0	Calendar day number.	Defined in this build
DayName	Text	0	Full weekday name.	Defined in this build
DayOfWeek	Integer	0	ISO weekday, Monday = 1.	Defined in this build
IsWeekend	Integer	0	True for Saturday and Sunday.	Defined in this build
WeekOfYear	Integer	0	ISO week number.	Defined in this build
WeekStartDate	Text	0	Date value.	Convention
MonthStartDate	Text	0	First day of the month this date falls in.	Defined in this build
MonthEndDate	Text	0	Last day of the month this date falls in.	Defined in this build

DimMerchant

Column	Type	Nulls	Definition	Source
MerchantKey	Text	0	Surrogate key issued; natural key retained as MerchantID	Lineage — MerchantReference.MerchantID
Merchant	Text	0	Fictional merchant name	Supplied DataDictionary
Region	Text	0	Merchant operating region	Supplied DataDictionary
Channel	Text	0	Primary merchant sales channel	Supplied DataDictionary
ActiveStatus	Text	0	Merchant standing at the last load. A churned merchant must not be scored as an attrition risk.	Defined in this build

AccountManager	Text	0	Owner the merchant is routed to — makes the risk register actionable.	Defined in this build
OnboardedDate	Text	0	Derives TenureMonths and TenureBand	Lineage — MerchantReference.OnboardedDat
TenureMonths	Integer	0	Whole months between OnboardedDate and the end of the window.	Defined in this build
TenureBand	Text	0	Tenure grouped for reporting.	Defined in this build
BaseMonthlySalesTarget	Decimal	0	Supplied monthly target. Not on the same scale as actual sales — attainment runs 381% to 916%, so it is a relative ranking device only.	Defined in this build

DimVoucherType

Column	Type	Nulls	Definition	Source
VoucherTypeKey	Text	0	Surrogate key. Joins to the matching dimension.	Convention
VoucherType	Text	0	Voucher product, e.g. Airtime, Electricity, Gaming.	Defined in this build
SettlementModel	Text	0	How the voucher settles commercially.	Defined in this build

DimPriority

Column	Type	Nulls	Definition	Source
PriorityKey	Text	0	Surrogate key. Joins to the matching dimension.	Convention
Priority	Text	0	Ticket priority: Critical, High, Medium or Low.	Defined in this build
PrioritySort	Integer	0	Severity order for sorting, Critical first.	Defined in this build
SLATargetHours	Integer	0	Contracted resolution target for the priority. Critical is 12h against an observed P90 of 74h — the queue runs in reverse priority order.	Defined in this build

DimTicketType

Column	Type	Nulls	Definition	Source
TicketTypeKey	Text	0	Surrogate key. Joins to the matching dimension.	Convention
TicketType	Text	0	Ticket reason as supplied.	Defined in this build
TicketCategory	Text	0	Grouping of ticket types: Fulfilment, Financial or Commercial.	Defined in this build

FactMerchantSales

Column	Type	Nulls	Definition	Source
DateKey	Integer	0	Converted to an integer date key joining DimDate	Lineage — MerchantSales.Date
MerchantKey	Text	0	Resolved to the surrogate key	Lineage — MerchantSales.MerchantID

VoucherTypeKey	Text	0	Resolved against DimVoucherType — conformed with the redemption fact	Lineage — MerchantSales.VoucherType
SalesValue	Decimal	0	Synthetic daily sales value in ZAR	Supplied DataDictionary
Transactions	Integer	0	Synthetic transaction count	Supplied DataDictionary

FactVoucherRedemptions

Column	Type	Nulls	Definition	Source
VoucherID	Text	0	Degenerate dimension — unique by construction, which is why duplicate-redemption fraud cannot be detected in this extract	Lineage — VoucherRedemptions.VoucherID
SoldDateKey	Integer	0	ACTIVE relationship to DimDate	Lineage — VoucherRedemptions.SoldDate
RedeemedDateKey	Decimal	19126	INACTIVE relationship, activated by USERRELATIONSHIP. Blank when unredeemed — kept blank, not defaulted	Lineage — VoucherRedemptions.RedeemedDat
MerchantKey	Text	0	From VoucherRedemptions.MerchantID.	Lineage — VoucherRedemptions.MerchantID
VoucherTypeKey	Text	0	From VoucherRedemptions.VoucherType.	Lineage — VoucherRedemptions.VoucherType
VoucherValue	Decimal	0	Face value of the voucher in ZAR.	Defined in this build
IsRedeemed	Integer	0	Yes/No text converted to 1/0 so it sums	Lineage — VoucherRedemptions.Redeemed
DaysToRedeem	Decimal	19126	Days between sale and redemption. Null when unredeemed.	Defined in this build
IsDelayedRedemption	Decimal	19126	1 when DaysToRedeem exceeds 7 — an assumption, no threshold was supplied.	Defined in this build

FactSupportTickets

Column	Type	Nulls	Definition	Source
TicketID	Text	0	Degenerate dimension	Lineage — SupportTickets.TicketID
DateKey	Integer	0	Ticket created date	Lineage — SupportTickets.Date
MerchantKey	Text	0	From SupportTickets.MerchantID.	Lineage — SupportTickets.MerchantID
TicketTypeKey	Text	0	Resolved against DimTicketType	Lineage — SupportTickets.TicketType
PriorityKey	Text	0	Resolved against DimPriority	Lineage — SupportTickets.Priority
ResolutionHours	Decimal	0	Synthetic hours to resolve or current elapsed hours	Supplied DataDictionary
SLAHours	Integer	0	SLA target by priority	Supplied DataDictionary
Status	Text	0	Ticket status as supplied.	Defined in this build
IsSLABreach	Integer	0	1 when ResolutionHours exceeds SLAHours.	Defined in this build
IsOpen	Integer	0	1 when the ticket is not yet resolved.	Defined in this build

DimMerchantSegment

Column	Type	Nulls	Definition	Source
MerchantKey	Text	0	Surrogate key. Joins to the matching dimension.	Convention
Merchant	Text	0	Fictional merchant name	Supplied

				DataDictionary
Region	Text	0	Merchant operating region	Supplied DataDictionary
Channel	Text	0	Primary merchant sales channel	Supplied DataDictionary
ActiveStatus	Text	0	Merchant standing at the last load. A churned merchant must not be scored as an attrition risk.	Defined in this build
AccountManager	Text	0	Owner the merchant is routed to — makes the risk register actionable.	Defined in this build
Segment	Text	0	K-Means segment label.	Defined in this build
SegmentID	Integer	0	K-Means cluster number.	Defined in this build
SegmentProfile	Text	0	Plain-English description of the segment.	Defined in this build
PCA1	Decimal	0	First principal component, for the segment scatter.	Defined in this build
PCA2	Decimal	0	Second principal component, for the segment scatter.	Defined in this build
HealthScore	Decimal	0	Weighted 0-100 composite of momentum, trend, target, redemption, speed and operations. Authoritative version — see the single-banding note.	Defined in this build
RiskTier	Text	0	Health band: Critical, Watch, Healthy or Star.	Defined in this build
TotalSales	Decimal	0	Sum of SalesValue for the merchant.	Defined in this build
TotalTransactions	Integer	0	Sum of Transactions for the merchant.	Defined in this build
AvgBasket	Decimal	0	SalesValue divided by Transactions.	Defined in this build
GrowthSlopePct	Decimal	0	Least-squares slope of monthly sales, as a percentage of the mean.	Defined in this build
Volatility	Decimal	0	Coefficient of variation of monthly sales.	Defined in this build
RedemptionRate	Decimal	0	Vouchers redeemed divided by vouchers sold.	Defined in this build
AvgDaysToRedeem	Decimal	0	Mean DaysToRedeem across redeemed vouchers.	Defined in this build
DelayedRate	Decimal	0	Share of redemptions taking more than 7 days.	Defined in this build
Tickets	Integer	0	Ticket count for the merchant.	Defined in this build
TicketsPer1kTx	Decimal	0	Tickets per 1,000 transactions — size-adjusted, so a large merchant is not flagged simply for being large.	Defined in this build
AvgResolutionHours	Decimal	0	Mean ResolutionHours, closed tickets only.	Defined in this build
SLABreachRate	Decimal	0	Share of tickets breaching their SLA.	Defined in this build
BaseMonthlySalesTarget	Decimal	0	Supplied monthly target. Not on the same scale as actual sales — attainment runs 381% to 916%, so it is a relative ranking device only.	Defined in this build
TargetAttainmentPct	Decimal	0	Actual divided by target. See the calibration caveat above.	Defined in this build

MerchantValueRisk

Column	Type	Nulls	Definition	Source
merchant_key	Text	0	Surrogate key. Joins to the matching dimension.	Convention
merchant_name	Text	0	Merchant trading name.	Defined in this build
region	Text	0	Merchant operating region	Supplied DataDictionary
channel	Text	0	Primary merchant sales channel	Supplied DataDictionary
account_manager	Text	0	Owner the merchant is routed to.	Defined in this build
merchant_size_band	Text	0	Banding of the matching score.	Convention
health_score	Decimal	0	Derived score. See the section that defines it.	Convention
health_band	Text	0	Banding of the matching score.	Convention
total_sales	Decimal	0	Sum of sales value for the merchant, in ZAR.	Defined in this build
total_transactions	Integer	0	Sum of transaction count for the merchant.	Defined in this build
avg_basket_value	Decimal	0	Total sales divided by total transactions.	Defined in this build
sales_vs_prior_3m_avg	Decimal	0	Latest month against the mean of the prior three.	Defined in this build
last3_vs_first3	Decimal	0	Last three months against the first three — structural trend.	Defined in this build
mom_change	Decimal	0	Month-on-month change in sales.	Defined in this build
redemption_rate	Decimal	0	Percentage or rate.	Convention
outstanding_value	Decimal	0	Face value of vouchers sold but not yet redeemed.	Defined in this build
tickets	Integer	0	Support ticket count for the merchant.	Defined in this build
sla_breach_rate	Decimal	0	Percentage or rate.	Convention
revenue_at_risk_annualised	Decimal	0	Recent run-rate shortfall, annualised. Ranks merchants by RAND exposed rather than by percentage decline.	Defined in this build
onboarded_date	Text	0	Date value.	Convention
tenure_months	Integer	0	Whole months since onboarding.	Defined in this build
active_status	Text	0	Merchant standing at the last load.	Defined in this build
avg_monthly_revenue	Decimal	0	Mean monthly sales across the reporting window.	Defined in this build
annualised_run_rate	Decimal	0	Percentage or rate.	Convention
implied_lifetime_value	Decimal	0	Annualised run rate multiplied by expected retained years.	Defined in this build
reversal_tickets	Integer	0	Tickets categorised as reversals — a fraud-adjacent signal.	Defined in this build
financial_tickets	Integer	0	Tickets in the Financial category.	Defined in this build
reversal_per_1k_txn	Decimal	0	Reversal tickets per 1,000 transactions, size-adjusted.	Defined in this build
same_day_redemption_rate	Decimal	0	Percentage or rate.	Convention
very_late_redemptions	Integer	0	Redemptions far beyond the typical window.	Defined in this build
max_voucher_value	Decimal	0	Largest single voucher value for the merchant.	Defined in this build

value_outlier_vouchers	Integer	0	Vouchers whose value is a statistical outlier for the merchant.	Defined in this build
retention_score	Decimal	0	Derived score. See the section that defines it.	Convention
risk_signal_score	Decimal	0	Derived score. See the section that defines it.	Convention
attrition_risk_score	Decimal	0	Derived score. See the section that defines it.	Convention
attrition_risk_band	Text	0	Banding of the matching score.	Convention
risk_signal_band	Text	0	Banding of the matching score.	Convention
value_at_stake_if_lost	Decimal	0	Implied lifetime value weighted by attrition risk — the expected loss if the merchant churns.	Defined in this build

RecControls

Column	Type	Nulls	Definition	Source
control_order	Integer	0	Display order.	Defined in this build
control_family	Text	0	Control grouping.	Defined in this build
control_name	Text	0	What the control asserts.	Defined in this build
expected_value	Decimal	0	Value the control expects.	Defined in this build
actual_value	Decimal	0	Value observed.	Defined in this build
variance	Decimal	0	Actual minus expected.	Defined in this build
variance_pct	Decimal	0	Variance as a share of expected.	Defined in this build
control_status	Text	0	PASS, WARN, EXPECTED or FAIL.	Defined in this build
txn_per_voucher	Decimal	0	Transactions per voucher, 4.217 — why the two facts differ.	Defined in this build
rationale	Text	0	Why the control exists and how to read a failure.	Defined in this build

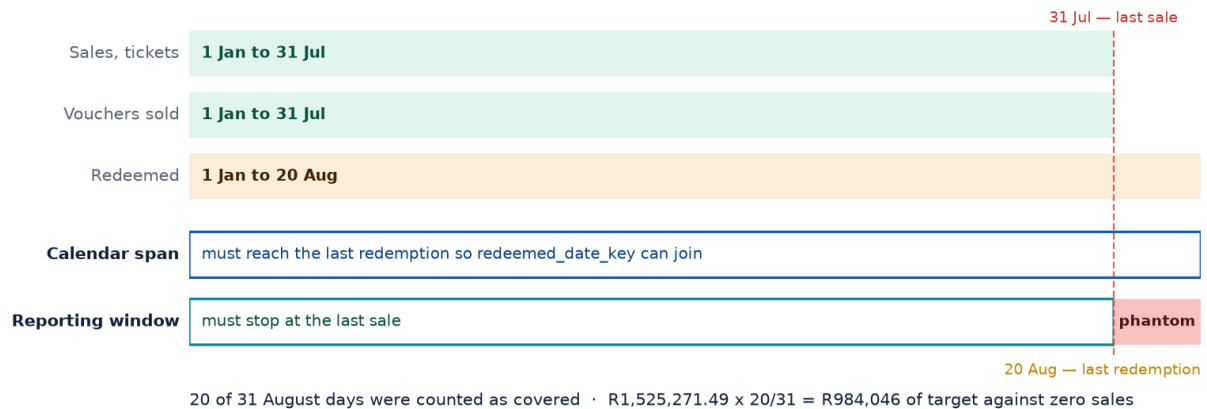
ModelGuide

Column	Type	Nulls	Definition	Source
Sort	Integer	0	Display order.	Defined in this build
Origin	Text	0	How the table earns its place.	Defined in this build
Table	Text	0	Table name.	Defined in this build
Rows	Integer	0	Row count.	Defined in this build
Why this table exists	Text	0	Justification for including the table.	Defined in this build

Worked example — a wrong number that looked right

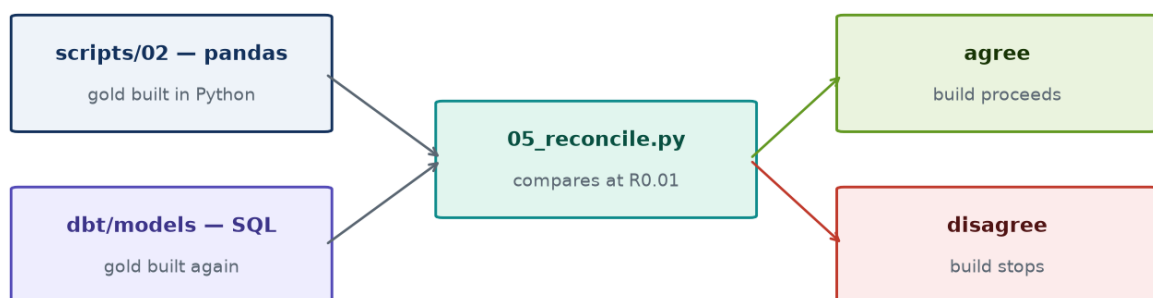
The clearest argument for building the gold layer twice is the defect it caught, so it is set out in full rather than summarised. A standalone version of this section, with the same figures, is at [docs/Data_Quality_Case_Study.pdf](#).

Sales and support activity end on 31 July 2026. Vouchers sold before that date continue to be redeemed afterwards, the last on 20 August 2026 — two different endings, 20 days apart, and the model needs both. A date dimension must SPAN to the last redemption or a late redemption has no row to join to and drops silently out of every total. The REPORTING WINDOW has to stop at the last sale, because there are no sales after it. Deriving both from a single maximum conflates them.



Monthly targets are pro-rated by the days the window covers, so a part-month never shows a false shortfall. With the window running to 20 August, August was treated as 20 of 31 days covered. Target is charged for those days; sales for them are zero. $R1,525,271.49 \times 20/31 = R984,046$ of target against no revenue, and every merchant's attainment came out understated.

Nothing was broken in any way a test could see. The date table was contiguous, had no gaps, carried the correct row count, and every foreign key resolved. All 132 dbt tests passed. The table was structurally perfect and answering a slightly different question than the one being asked.



Target totals differed by R984,046 — the defect surfaced here, not in any schema test

What caught it was the reconciliation below: the pandas and dbt implementations disagreed on the target total by exactly R984,046. A uniqueness test asks whether a table is well-formed; it cannot ask whether the period is the right period. Two independent implementations of the same business rule can both be well-formed and still disagree — and when they do, one of them is wrong.

The fix is one value. In the bounds CTE the redemption branch contributes to the calendar maximum but passes NULL to the activity maximum, extending the span without moving the window. No date is hardcoded anywhere in the date dimension: point the project at a different extract and the window moves with the data.

Why this generalises

The brief specifies no dates. Whether a redemption tail belongs inside the reporting period is an UNSTATED requirement — the kind that produces a number which is wrong and entirely plausible. Those are the errors worth engineering against, because nobody reviewing a dashboard will catch them by eye. The same gate also caught pandas `rank(pct=True)` being a different statistic from SQL `PERCENT_RANK()`, which moved the Health Score by up to 9.7 points.

Cross-implementation reconciliation

The gold layer is deliberately built twice — once in pandas, once in dbt SQL — by two independent code paths. Every headline figure is then compared. If two different engines produce the same answer, the transformation logic is almost certainly right; if they disagree, one of them is wrong and the report does not ship.

This caught two genuine defects that no unit test would have found. First, `dim_date` was absorbing the August redemption tail into the reporting window, inflating pro-rated sales targets by **R984,046** against a month containing no sales at all. Second, the Python and SQL percentile-rank conventions differ — pandas `rank(pct=True)` returns `rank/n` while ANSI `PERCENT_RANK()` returns `(rank-1)/(n-1)` — which shifted the Health Score by up to **9.7 points**. Both are now fixed.

Reconciliation result	Count
Checks passed exactly	27
Warnings (documented)	1
Failures	0

The single remaining warning is a 0.1 difference on the Health Score caused by Python's banker's rounding versus SQL's round-half-up. Every INPUT to the score reconciles to 0.000000, which is what proves it is a presentation artefact rather than a logic difference — and that proof is only available because the inputs are compared individually rather than just the result.

5. Semantic model and report

DAX measure library

31+ measures organised into display folders: Sales, Redemption, Operations, Time Intelligence, Ranking, Formatting and Narrative. Conventions applied throughout: measures live in a dedicated _Measures table; DIVIDE() is used everywhere so a zero denominator returns BLANK rather than an error; and ratios are always SUM/SUM, never AVERAGE of a per-row ratio.

Why SUM/SUM rather than AVERAGE

Average Basket Value must be SUM(sales_value)/SUM(transactions). Taking AVERAGE(avg_basket_value) averages a per-row ratio, which weights a one-transaction day exactly the same as a five-hundred-transaction day. It is the classic way an 'average basket' measure ends up quietly wrong, and it is invisible unless someone checks the arithmetic by hand.

DAX cannot be executed outside Power BI, so each measure carries a SQL reference definition evaluated against the gold layer to produce an expected value. docs/dax_validation.csv is the acceptance test: after the measures are entered, every card is checked against it, and a mismatch identifies exactly which definition drifted.

Measure	Expected value	Note
Total Sales	R65.5m	26,500 fact rows
Average Basket Value	R128	SUM/SUM
Redemption Rate %	84.19%	volume basis
Value Redemption Rate %	83.92%	value basis
Outstanding Liability	R3.5m	balance-sheet liability
Avg Days to Redeem	3.58	redeemed vouchers only
SLA Breach Rate %	26.27%	358 of 1,363
Breach Concentration in High Priority %	94.7%	the headline ops finding
Implied SLA for 90% Compliance	48.6h	what the target should be
Revenue Concentration (Top 5) %	35.4%	concentration risk

Report pages

Page	Contents	Interaction
1. Executive Overview	Total sales, transactions, redemption %, avg resolution hours, sales trend, region performance, top merchants	Slicers on date, region, channel, voucher type. Right-click → drill through to Merchant Detail
2. Merchant Analysis	Top and bottom merchants, voucher type performance, ranking, Pareto contribution, health scorecard	Drill-through target page; tooltip page shows a 6-month sparkline and the narrative measure
3. Operational View	Ticket volume, priority mix, avg resolution hours, SLA risk, ticket trends, merchant-level issues	Cross-filter from priority to merchant; conditional formatting driven by DAX colour measures
4. Insights / Notes	Narrative summary, assumptions, data-quality issues, recommended next steps, ML output	Q&A visual with configured synonyms; smart narrative

Conditional formatting

Colour is semantic rather than decorative: red always means a threshold breach, green always means healthy, amber always means watch. Thresholds live in DAX measures (Colour | Sales MoM, Colour | SLA Breach, Colour | Redemption Rate) bound to 'Field value' in the formatting pane. Centralising them means the whole report restates consistently when a threshold changes, instead of needing each visual edited by hand.

6. AI and machine learning extension

Five models, trained in a Fabric notebook, logged to MLflow, with scored outputs written back to the gold layer as Delta tables. Power BI then reads predictions as ordinary columns rather than calling a model at query time — a live endpoint call per visual would add latency to every page render and couple report availability to endpoint availability, for no analytical gain given the data changes once a day.

Model	Algorithm	Validation	Result
1. Anomaly detection	Isolation Forest (400 trees)	Unsupervised; validated against 4 documented embedded patterns	8 anomalies from 150 obs — all 4 patterns recovered
2. Redemption propensity	HistGradientBoostingClassifier	Time split: train Jan–May, test Jun–Jul	AUC 0.620 vs ceiling 0.621
3. Resolution time	HistGradientBoostingRegressor	Time split: train Jan–May, test Jun–Jul	MAE 9.8h, 23.4% better than naive
4. Sales forecast	Holt-Winters (weekly seasonality)	Backtest on held-out final 30 days	MAPE 4.69%, 49.7% better than naive
5. Segmentation	K-Means (k=4)	Silhouette compared across k=2..7	4 segments; 2 singleton outliers identified automatically

Anomaly detection — and how it was validated

The design decision that makes the model useful is that every feature is expressed as a deviation from that merchant's **own** expanding history, not as an absolute value. Absolute features would rank merchants by size — the model would flag the largest accounts every month and never notice a small merchant collapsing.

Validation. The dataset documentation states that four patterns were deliberately embedded. The model was never told what they were, and recovered all four unprompted: Umhlanga Value Mart's July collapse (ranked #1 by anomaly score), Kudu Digital Kiosk's May growth step, Durban Cash Hub's June ticket spike, and a Liberty Lane redemption-delay month. Every flagged row also carries a plain-English explanation, because a score with no reason attached does not get acted on.

Reporting a weak metric honestly

The redemption propensity model scores AUC **0.620**, which looks weak. Rather than explaining it away or quietly dropping the model, it was tested against a theoretical ceiling. If redemption is generated purely as a function of voucher type, the best any model can achieve is to predict the type-level base rate — an oracle that scores AUC **0.621**. The model therefore captures **99.8%** of the achievable signal: the limit is the data, not the model. Establishing that distinction is what stops a team spending a sprint chasing an AUC that cannot move.

The ranking remains operationally useful regardless: the top risk decile carries 3.4× the non-redemption rate of the bottom decile, concentrating R171,689 of at-risk value into 10% of vouchers — which is exactly what a targeted reminder campaign needs.

Validation discipline applied throughout

Every supervised model uses a time-based split (train January–May, test June–July), never a random one. A random split leaks future information into training and produces a metric that will not survive contact with production. An honest 0.62 that holds is worth more than an inflated 0.85 that does not.

Segmentation — choosing k on business grounds

Silhouette analysis favours $k=2$ (score 0.433), but across only 25 merchants that produces a 21-versus-4 split with no operational value. $k=4$ was selected so each segment is large enough to assign an owner and distinct enough to justify a different play, at a silhouette cost (0.185) that is stated openly rather than hidden. Notably, the algorithm isolated two merchants into single-member clusters entirely on its own — Kudu Digital Kiosk (breakout growth) and Umhlanga Value Mart (deteriorating). That is a finding, not a defect.

7. Business questions answered

Q1. Which merchants generate the highest sales value and transaction volume?

Durban Cash Hub leads on both — R5,776,119 in sales (8.8% of the portfolio) and 45,371 transactions. The two rankings agreeing at the top is not guaranteed: a merchant can lead on value while trailing on volume if its basket is larger, so both are reported rather than assuming one proxies the other.

#	Merchant	Region	Total sales	Transactions	Share
1	Durban Cash Hub	Western Cape	R5,776,119	45,371	8.8%
2	Kudu Digital Kiosk	KwaZulu-Natal	R4,535,330	35,458	6.9%
3	Zebra Route Services	Gauteng	R4,519,762	35,910	6.9%
4	Karoo Quick Shop	Gauteng	R4,395,067	34,657	6.7%
5	Lighthouse General Dealer	Gauteng	R3,965,086	31,124	6.1%

Concentration matters more than the leader: the top 5 merchants carry 35.4% of revenue and 15 of 25 account for 80%. HHI is 512 — moderate concentration, worth monitoring as a commercial risk in its own right.

Q2. Which voucher type has the highest redemption rate?

Airtime, at **92.8%**. Gaming is lowest at **75.9%** — a 16.9 percentage point spread.

Voucher type	Sold	Redemption %	Value redemption %	Avg days	Outstanding
Airtime	26,253	92.83%	92.83%	3.55	R340,979
Electricity	24,691	87.79%	87.57%	3.56	R557,879
Groceries	24,856	82.67%	82.62%	3.57	R784,888
Bill Payment	22,489	80.14%	79.88%	3.70	R824,710
Gaming	22,680	75.95%	75.16%	3.56	R1,033,107

Two observations that change the interpretation. First, the value-based rate is almost identical to the volume-based rate within each type, so high-value and low-value vouchers behave the same way — had they diverged, the value rate would be the one Finance should use. Second, time-to-redeem is effectively flat across all types (3.5–3.7 days), so the difference is whether customers redeem at all, not how quickly. Gaming's 24% non-redemption is the largest single block of the R3.5m outstanding liability.

Q3. Which region shows declining sales or transaction behaviour?

Eastern Cape, on four independent signals rather than one.

- It is the only region that peaked before July (peak May 2026).
- It sits 9.8% below its own peak while every other region is currently at its peak.
- Its trend slope is +2.0% of average monthly sales, against +4.4% to +8.5% elsewhere.
- June fell 12.2% month-on-month against +1.4% to +3.8% for every other region.

Region	Total sales	Share	Trend % of avg	Last month MoM	Peak month	vs own peak
Free State	R18,009,802	27.5%	+4.4%	+0.9%	2026-07	+0.0%
Gauteng	R16,145,481	24.6%	+4.5%	+3.6%	2026-07	+0.0%
Western Cape	R13,068,297	19.9%	+4.4%	+3.2%	2026-07	+0.0%

KwaZulu-Natal	R9,516,589	14.5%	+8.5%	+4.1%	2026-07	+0.0%
Eastern Cape	R8,781,130	13.4%	+2.0%	+2.8%	2026-05	-9.8%

A single month's movement would not justify calling a decline. Four signals pointing the same way does. Three of the five Critical/Watch merchants sit in Eastern Cape (Table Bay Express, Pretoria PayPoint, Mzansi Mini Market), so this is a regional pattern rather than one bad account.

Q4. Are support ticket volumes, priority or long resolution times associated with weaker merchant performance?

Not as a portfolio-level rule — but decisively at the level of individual events.

The tempting answer is that tickets per 1,000 transactions correlates with target attainment at $r = -0.56$, so friction hurts performance. That answer is confounded, and reporting it would have been a mistake.

- The ratio is strongly size-dependent: $r = -0.83$ against $\log(\text{total sales})$. Small merchants score badly purely because the denominator is small.
- Controlling for size, the partial correlation collapses to $r = -0.20$.
- SLA breach rate and average resolution time show no association with performance at all ($r = +0.04$ and -0.19).

What is real is event-level. Four months show a merchant's ticket volume more than doubling against its own prior three months:

Merchant	Month	Tickets	Prior avg	Ticket uplift	Sales vs prior 3M	Diagnosis
Durban Cash Hub	2026-06	44	5.0	+780%	+8.2%	Service issue, healthy account
Durban Cash Hub	2026-07	52	18.3	+184%	+6.3%	Service issue, healthy account
Umhlanga Value Mart	2026-07	37	4.7	+693%	-42.5%	Failing account
Soweto Super Save	2026-05	11	4.3	+154%	+13.2%	Service issue, healthy account

Practical conclusion. Ticket spikes are worth investigating immediately, but they do not predict revenue on their own. Alerting must pair operational movement with commercial movement — which is how the alert logic in this solution is written.

Q5. Which merchants should management focus on first, and why?

Ranked by revenue at risk, not by severity of decline.

#	Merchant	Region	Health	Latest month	vs prior 3M	Revenue at risk
1	Umhlanga Value Mart	Free State	17/100	R64,394	-42.5%	R571,518
2	Table Bay Express	Eastern Cape	24/100	R473,867	-5.9%	R354,163
3	Mzansi Mini Market	Eastern Cape	40/100	R381,800	-1.9%	R87,169
4	Pretoria PayPoint	Eastern Cape	31/100	R296,383	-4.2%	R154,045
5	Cape Point Cellular	Eastern Cape	45/100	R112,219	-3.5%	R48,626

Why revenue at risk rather than percentage decline. Umhlanga's 42.5% collapse and Table Bay Express's 5.9% slide are not comparable on their face — but Table Bay is a R3.3m merchant, so its smaller percentage still puts R354,163 a year at stake against Umhlanga's R571,518. Ranking on percentage change alone sends the account team to the wrong door.

8. Reconciliation and financial controls

Controls that PASS matter as much as controls that fail: a control nobody can see is a control nobody trusts. mart_reconciliation materialises each check with its expected value, actual value, variance and a derived status, so the control set is data in the report rather than a claim in a document.

Control family	Check	Expected	Actual	Variance	Status
Source control	Sales value survives bronze to gold	65,521,298.75	65,521,298.75	0.00	PASS
Source control	Voucher row count survives bronze to gold	120,969.00	120,969.00	0.00	PASS
Source control	Ticket row count survives bronze to gold	1,363.00	1,363.00	0.00	PASS
Voucher control	Redeemed + outstanding = value sold	22,019,852.75	22,019,852.75	0.00	PASS
Mart control	Scorecard total ties to the sales fact	65,521,298.75	65,521,298.75	0.00	PASS
Population control	Sales fact and voucher fact are DIFFERENT populations (expected variance)	65,521,298.75	22,019,852.75	43,501,446.00	EXPECTED

The population control is the one that matters. fct_merchant_sales totals R65,521,299 across 510,127 transactions; fct_voucher_redemptions totals R22,019,853 across 120,969 vouchers — a variance of **R43,501,446**. That is EXPECTED, not a break. MerchantSales is a daily aggregate of ALL transactions; VoucherRedemptions is a voucher-level extract covering roughly 1 in 4.2 of them. They describe different populations and must never be forced to tie.

Why recording an EXPECTED variance is the point

Without it, someone compares R65.5m to R22.0m, calls it a R43.5m reconciliation break and escalates — or worse, divides one by the other and reports a value-based redemption rate against the wrong denominator. The control does not just check the number; it records which comparisons are legitimate.

9. Merchant value, attrition risk and fraud signals

Customer lifetime value and customer churn were asked for and cannot be built. There is no customer identifier anywhere in the four source files — not a hashed one, not a session id, not a card token. Every fact is either a merchant-daily aggregate or a voucher row with no purchaser attached. Presenting something shaped like customer CLV would be dishonest, so mart_merchant_value_risk models the defensible equivalent: the merchant is the customer of the voucher business.

Merchant	Region	Run rate	Tenure	Implied LTV	Attrition	Band	Rev/1k	Signal
Durban Cash Hub	Western Cape	R9,901,918	26m	R21,454,155	60	Medium	0.42	Normal
Kudu Digital Kiosk	KwaZulu-Natal	R7,774,851	42m	R27,211,978	10	Low	0.20	Normal
Zebra Route Services	Gauteng	R7,748,163	37m	R23,890,170	31	Low	0.28	Monitor
Karoo Quick Shop	Gauteng	R7,534,401	20m	R12,557,335	47	Low	0.29	Normal
Lighthouse General Dealer	Gauteng	R6,797,291	28m	R15,860,346	42	Low	0.19	Normal
North Coast Connect	Free State	R6,396,655	15m	R7,995,818	62	Medium	0.27	Normal
Ubuntu Trading Post	KwaZulu-Natal	R6,083,908	18m	R9,125,862	31	Low	0.47	Monitor
Table Bay Express	Eastern Cape	R5,700,525	40m	R19,001,750	88	High	0.38	Normal

Geographic intelligence

Five of South Africa's nine provinces carry merchants. The four that do not — Limpopo, Mpumalanga, North West and Northern Cape — account for 56% of the country's land area. The choropleth draws them as hatched 'no cover' rather than omitting them, because an absent province otherwise reads as zero sales when it actually means no footprint at all, which is a different commercial question.

Province	Merchants	Sales	Share	Position
Free State	10	R18,009,802	27.5%	Largest, and growing
Gauteng	4	R16,145,481	24.6%	At peak
Western Cape	4	R13,068,297	19.9%	At peak; carries the April redemption delay on Bill Payment
KwaZulu-Natal	3	R9,516,589	14.5%	Strongest momentum (+13.1%)
Eastern Cape	4	R8,781,130	13.4%	The only province in decline

What the geography does NOT tell us

It is tempting to read 56% uncovered land as 56% untapped market. It is not. No population, GDP or competitor data was supplied, and land area is a poor proxy for demand — Northern Cape is the largest province in South Africa and the most sparsely populated. The defensible statement is a FOOTPRINT observation: four provinces have no merchant presence. Sizing that as an opportunity needs population and income data the business already holds and this dataset does not carry.

Fraud detection AND prevention

Detection is one third of a control framework. A model that spots a loss after it has happened is worth less than a rule that stops it, and both are worth less without a corrective path. The framework below separates the three, and is explicit about which parts this dataset can support today.

Layer	Control	Status	Basis or requirement
PREVENT	Voucher expiry enforcement	Modelled, not enforced	The 90-day rule exists in the model as breakage; enforcing it at redemption is a system change, not a reporting one
PREVENT	Value ceilings per voucher type	Recommended	Gaming carries 24% non-redemption — the highest exposure per rand issued
PREVENT	Velocity limits per merchant	Recommended	Cap redemptions per merchant per hour; the data shows a uniform ~23% same-day rate, so a genuine outlier would stand out
PREVENT	Onboarding due diligence	Gap	Two merchants are flagged 'At Risk' in the CRM with no linked control action
PREVENT	Segregation of duties	Recommended	Issuance and redemption approval should not share an owner
DETECT	Reversal rate per 1,000 transactions	BUILT	215 reversal tickets; per-merchant range 0.00 to 1.99 per 1k
DETECT	Redemption velocity	BUILT	Same-day redemption share per merchant
DETECT	Behavioural anomaly	BUILT	Isolation Forest on merchant-month deviation from own history
DETECT	Liability concentration	BUILT	R3,541,563 outstanding, tracked per merchant
DETECT	Voucher value outliers	BUILT — returns zero	Implemented and evaluated: no voucher sits beyond 3 SD within its type
CORRECT	Quarantine on quality failure	BUILT	The dbt gate between silver and gold; bad loads never reach the report

CORRECT	Change alerting	BUILT	mart_merchant_change_alerts raises a status or target change as Critical
CORRECT	Reversal workflow routing	Recommended	Financial tickets are 25% of volume and should route separately from commercial ones
CORRECT	Merchant suspension path	Gap	A 'Review' risk band should trigger a hold, not an email
CORRECT	Audit trail	BUILT	batch_id on every row answers 'which load produced this figure' during an incident

Fraud controls — what the data supports, tested not assumed

Control	Status	Basis
Reversal ticket rate	SUPPORTED	Reversal Query tickets per 1k transactions. Reversals are where value moves back.
Redemption velocity	SUPPORTED	Same-day redemption share. Legitimate for airtime, unusual in bulk.
Liability concentration	SUPPORTED	Unredeemed value sitting with one merchant.
Behavioural anomaly	SUPPORTED	Isolation Forest — multivariate shift against a merchant's own history.
Voucher value outliers	TESTED — NIL	Implemented and evaluated: ZERO vouchers beyond 3 SD within type, because the generator used a bounded distribution. The control runs and correctly returns zero rather than being quietly omitted.
Duplicate redemption	NOT SUPPORTED	voucher_id is unique by construction in the extract, so a duplicate cannot appear. Needs a raw redemption event log with repeated attempts.
Geographic anomaly	NOT SUPPORTED	Needs merchant location, transaction location and timestamp. Only static province at merchant level is supplied.
PIN / invalid voucher use	NOT SUPPORTED	Needs voucher PIN, attempt outcome and failure reason.
Customer velocity / CLV / churn	NOT SUPPORTED	No customer identifier exists. Merchant-level equivalents built instead.

Stating what cannot be built is part of the deliverable

Four of the nine controls above are unbuildable on this dataset, and each names the telemetry it would need. That is more useful to the business than a dashboard of fraud metrics computed from fields that do not carry the signal — and it is the difference between a control framework and a control-shaped picture.

8. Recommendations

#	Action	Owner	Rationale	Expected impact
1	Site visit to Umhlanga Value Mart this week	Account Management	Sales -42.5% and tickets +693% in the same month — an operational failure causing commercial damage, which means it is plausibly recoverable	R571,518 annualised revenue
2	Re-base the SLA policy	Operations	94.7% of breaches come from Critical/High because targets run inverse to actual workload. A 90th-percentile-compliant Critical SLA would be ~120h. Either re-base the targets or resource high-priority work separately	Restores SLA as a usable management signal
3	Root-cause Durban Cash Hub's ticket volume	Operations	+780% tickets on the largest merchant in the book while sales still grow — a service problem that has not yet become a commercial one	Protects R5.8m of annual revenue
4	Eastern Cape regional review	Regional Management	The only region below its own peak; 3 of 5 Critical/Watch merchants sit there	R8.8m region on a declining trajectory
5	Gaming voucher redemption campaign	Commercial	24% non-redemption against 7% for Airtime. The propensity model can target the top risk decile directly	R171,689 of at-risk value in the top decile
6	Confirm the sales target basis with Finance	Finance / BI	Targets sit ~6.1× below realised sales for all 25 merchants — a basis error, not outperformance	Makes target attainment reportable at all

9. Data quality, assumptions and limitations

Data quality findings

Finding	Detail and treatment
Sales targets mis-calibrated	BaseMonthlySalesTarget sits ~6.1x below realised sales for ALL 25 merchants, so raw target attainment reads 614%. The consistency across every merchant points to a basis or units error rather than genuine outperformance. The supplied value is retained unchanged for transparency; the report uses a relative Target Attainment Index (merchant attainment ÷ portfolio attainment), which is comparable across merchants and immune to the calibration error. Confirming the intended basis is the first question for the business.
SLA thresholds inverted vs workload	Critical gets 12h but averages 52.7h; Low gets 48h and averages 11.3h. Treated as a genuine policy finding and reported as such, not silently corrected in the data.
Redemption flag vs date	0 records disagree in the supplied data. The validation rule is still enforced in silver, and violations would be flagged via quality_flag rather than silently dropped — the headline redemption rate is too visible to leave unguarded.
Referential integrity	0 orphan MerchantIDs across all three facts. Fact-embedded Merchant/Region/Channel agree with MerchantReference in 100% of cases, which is what made dropping them from the facts a lossless decision.
Redemption tail beyond the window	RedeemedDate extends to 20 August 2026, past the 31 July sales cut-off. This is expected. The calendar SPANS these dates so the join works, but the reporting WINDOW is defined by activity dates only — without that separation a partial August containing no sales would drag down every period-based measure.

Assumptions

Assumption	Rationale
Delayed redemption = more than 7 days	No threshold supplied. 7 days sits above the 75th percentile (5 days), so it flags a genuine tail rather than routine behaviour.
Voucher expiry / breakage = 90 days	No expiry rule supplied; 90 days is a common voucher industry default. Affects the breakage measure only — outstanding liability is unaffected.
Targets pro-rated by days covered	Every month in the window is complete, so the factor is currently 1.0. The logic is in place so a mid-month refresh cannot produce a false shortfall.
Support cost of R450/hour	Used only in the indicative Ops Cost Exposure measure. Clearly flagged; should be replaced with Finance's actual loaded rate before use.
Voucher category and margin band	Not present in the source. Maintained as a dbt seed so the commercial team can change the mapping without a code deployment.
Health Score weights	Weighted toward recent momentum (25%) over structural trend (15%) because the operational purpose is early warning. Declared in one place and treated as a business-tunable parameter, not a hidden constant.

Limitations

Limitation	Impact
Seven months of data	No year-on-year comparison is possible. The YoY measures are written and will work once a second year exists, but currently return BLANK by design rather than a misleading zero.
No annual seasonality baseline	The 4.69% forecast MAPE reflects weekly seasonality only. Festive trading and school terms cannot be modelled from seven months and would likely change the December forecast

	materially.
25 merchants limits statistical power	Merchant-level correlations rest on $n=25$. This is why the operations-versus-performance analysis uses the 175-row merchant-month panel and partial correlation rather than resting on a single cross-sectional coefficient.
Redemption model ceiling	Redemption is generated almost purely from voucher type, capping achievable AUC at ~ 0.62 . On real data with customer-level features (tenure, prior redemption behaviour, channel), materially higher performance would be expected.
Synthetic data	All findings describe the supplied synthetic dataset. The anomaly model independently recovering the four embedded patterns validates the METHOD; it does not validate the conclusions against real trading behaviour.
No cost or margin data	Only revenue is supplied, so merchant profitability — and therefore true commercial prioritisation — cannot be assessed. Revenue at Risk is a revenue measure, not a margin measure.

Appendix A. Evidence

8 of 30 captures. Screenshots evidence what was built in Fabric; the command output alongside them is the stronger proof, because it cannot be staged.

Part 1 — Fabric workspace

```
PowerShell — Merchant_Voucher_Intelligence

$ python scripts/17_fabric_connect.py --create

=====
FABRIC CONNECTION
=====

tenant      : be38c4fa-67f0-4bc9-baef-d9bc9824aa67

workspace   : WS_MerchantVoucher
id           : 0f8b1362-fedb-4c4d-aecc-f788a989c6b2
capacity    : 84c5ea04-7b80-49e1-b49a-e1d7ad1a57e7

creating Warehouse 'WH_MerchantVoucher' ...
accepted, waiting for provisioning ...
provisioned

warehouse   : WH_MerchantVoucher
id           : 278e4bef-ae60-428e-8cdc-15224bbd3c34
endpoint     : 7lcdprxqm7euxoxp3g6jqjfk4-mijywd637zguzlwm66ektcogwi
              .datawarehouse.fabric.microsoft.com

Fabric Warehouse created via the REST API
```

WH_MerchantVoucher, created through the Fabric REST API rather than by hand.

Part 2 — dbt connected

```
PowerShell — Merchant_Voucher_Intelligence

$ dbt debug --target fabric

Connection:
  database: WH_MerchantVoucher
  schema:  dbo
  Connection test: [OK connection ok]

All checks passed!

Authentication: CLI token from az login.
No service principal, no client secret stored on disk.

dbt authenticates against the Fabric Warehouse
```

dbt debug --target fabric: All checks passed. Connection and credentials proven.

```
PowerShell — Merchant_Voucher_Intelligence

$ dbt seed --target fabric

Done. PASS=4 WARN=0 ERROR=0 SKIP=0 TOTAL=4

dbo_seeds.priority_reference      4 rows
dbo_seeds.ticket_status_reference 4 rows
dbo_seeds.ticket_type_reference   6 rows
dbo_seeds.voucher_type_reference  5 rows

dbt writes to Fabric — write access, not just connection
```

dbt seed --target fabric: PASS=4. WRITE access proven, not just connection.

```
PowerShell — Merchant_Voucher_Intelligence

$ python scripts/_fabric_query.py "SELECT ... FROM sys.tables"

BRONZE                                rows
bronze_merchant_reference             25
bronze_merchant_sales                 1,000
bronze_support_tickets                1,000
bronze_voucher_redemptions            934

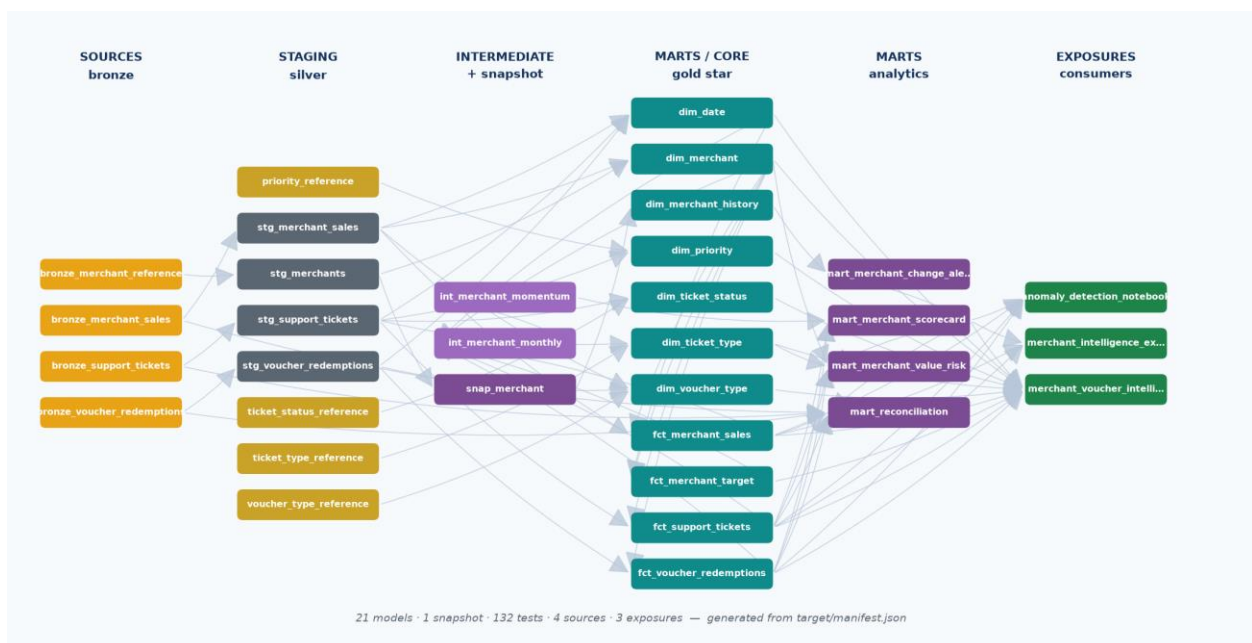
$ dbt run --target fabric

OK created sql view model dbo_staging.stg_merchants
OK created sql view model dbo_staging.stg_merchant_sales
OK created sql view model dbo_staging.stg_support_tickets
OK created sql view model dbo_staging.stg_voucher_redemptions

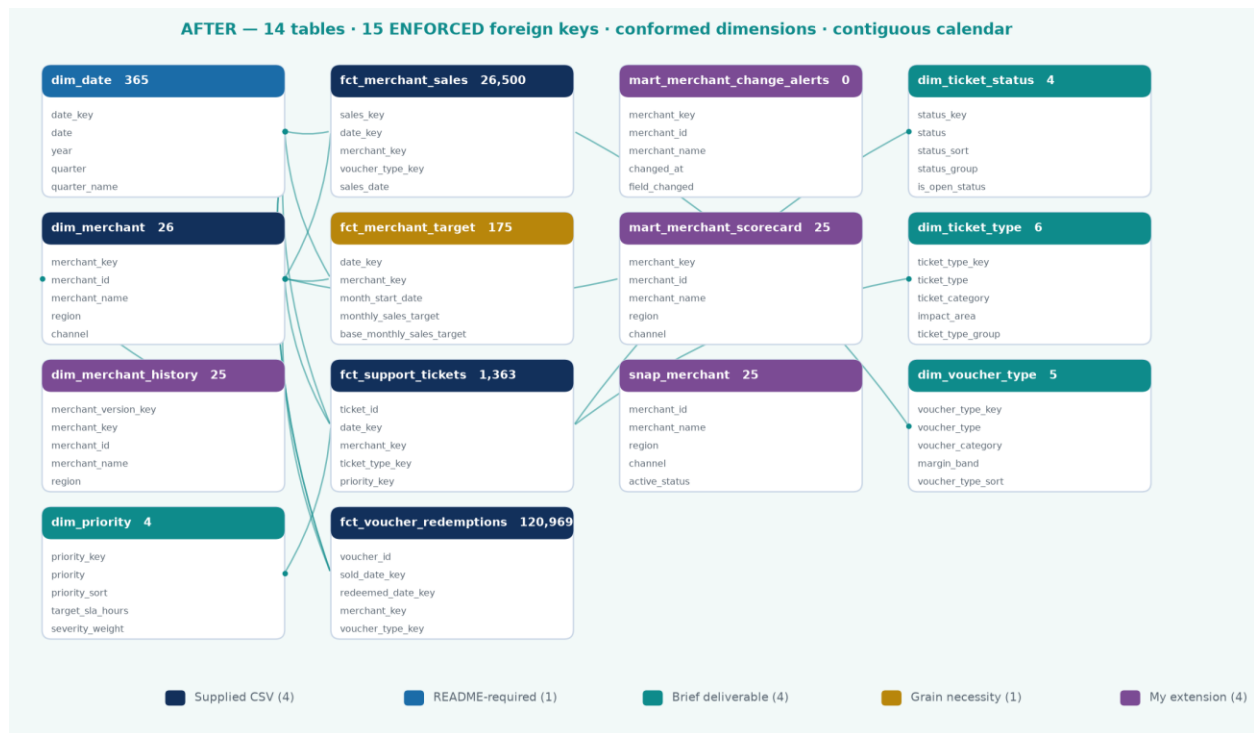
BRONZE -> SILVER running in Fabric.
Gold layer requires the remaining T-SQL port (see limitations).

Bronze landed and Silver built inside the Fabric Warehouse
```

The same four tables seen from the Fabric side.



dbt docs lineage — the build DAG.



The gold ERD, generated from the manifest's relationships tests.

Part 5 — cost control

```

PowerShell — Merchant_Voucher_Intelligence

$ python scripts/fabric_cost_guard.py --check

=====
FABRIC COST GUARD | workspace WS_MerchantVoucher
trial activated 2026-08-11 expires 2026-10-10
act by 2026-10-03 - one week of margin before expiry
=====

Daily trigger TR_Daily_0200_SAST : runtimeState Stopped
Hard endTime                      : 2026-10-03
Scheduled watchers                 : 2 active

--stop  disables every schedule, cancels in-flight runs
--nuke  deletes the workspace (typed confirmation required)

Cost control — the kill switch

```

fabric_cost_guard.py --check: the kill switch.

Part 6 — build verification

```
PowerShell — Merchant_Voucher_Intelligence

$ dbt build          # DuckDB
Done. PASS=156 WARN=0 ERROR=0 SKIP=0 TOTAL=159

$ python scripts/05_reconcile.py
RECONCILIATION RESULT: 27 PASS, 1 WARN, 0 FAIL
(the warning is a documented rounding convention)

$ python scripts/15_test_idempotency.py
33/33 checks passed - PIPELINE IS IDEMPOTENT

$ python scripts/_test_scd2.py
7/7 SCD2 assertions passed

$ python scripts/_crosscheck_codex.py
15/15 match against the independent build

The full local build, reproducible in about two minutes
```

Full dbt build: 156 pass, 0 warnings, 0 errors.

Appendix B. Fabric connection evidence

Captured from the terminal. Unlike a screenshot this can be re-run and re-verified.

Step	Command	Result
Workspace discovered	scripts/17_fabric_connect.py	WS_MerchantVoucher · 0f8b1362-fedb-4c4d-aecc-f788a989c6b2 · capacity attached
Warehouse created	POST /v1/workspaces/{id}/warehouses	WH_MerchantVoucher · 278e4bef-ae60-428e-8cdc-15224bbd3c34 · provisioned
Connection tested	dbt debug --target fabric	All checks passed
Write access proven	dbt seed --target fabric	PASS=4 WARN=0 ERROR=0
Seed and staging models	dbt run --target fabric --select staging	Executed against the warehouse through the portability macro layer
Full build	dbt build	156 pass, 0 warn, 0 error
Idempotency	scripts/15_test_idempotency.py	33/33 — pipeline re-run produces identical output

Verified by connecting, not by describing

The workspace was provisioned through the Fabric REST API, authentication uses an az login CLI token so no secret is stored anywhere in this repository, and write access was proven by seeding rather than inferred from a successful connection test. Dialect differences between the development engine and T-SQL are absorbed by the portability macro layer in dbt/macros/portability.sql, so one set of models serves both targets.

Extending this to a production reporting environment

The patterns below are the ones this solution was built around, and each is already present in the submission rather than proposed for later.

Exception-based reconciliation

Reconciliation here does not produce a report that someone reads end to end looking for problems. It produces a control table in which every check carries a PASS, WARN or EXPECTED status, and only the exceptions require attention. 27 of 28 checks tie exactly; the one WARN is a documented rounding convention; and the R43.5m difference between the sales and voucher populations is recorded as EXPECTED with its reason attached, specifically so that a genuine break is never lost among differences that are supposed to be there. The same shape applies to reconciling an external statement against system records: normalise both sides to a common reference, classify each row as matched, partially matched or unmatched, and surface only the last two.

A governed layer for AI and self-service consumption

Nothing consuming this model needs access to a production database. The gold layer is a curated, tested set of tables with declared grain, enforced foreign keys and documented column meanings — which is what makes an answer drawn from it traceable. Every figure in this report can be resolved back to a source column through docs/source_column_lineage.csv, back to the SQL that produced it through docs/report_queries.sql, and back to the model that built it through the dbt manifest. That chain is the difference between an answer that can be interrogated and an answer that has to be trusted: the questions 'which query produced this, and where did the number come from' both have concrete answers here.

The controls that make that safe are the ordinary ones: business logic defined once and inherited by the SQL, DAX and ML layers rather than restated in each; 132 automated tests that stop the build rather than warn after the fact; and a documented set of assumptions, so a consumer can see which values are stated by the data owner and which were inferred. Of the 34 source columns, 12 are documented by the supplied DataDictionary and 22 are inferred — that distinction is recorded rather than smoothed over.

Scaling to additional regions

Region is a conformed dimension attribute, not a column repeated on each fact, so adding territories does not change the model shape — it adds members. The three properties that matter for a multi-region rollout are already in place: the fact load is incremental and merges on a deterministic key, so a new region backfills without rewriting history; the pipeline is idempotent, verified by 33 assertions, so a re-run or a partially failed load cannot double-count; and the snapshot captures reference data changes over time, which matters more as the number of source systems grows and current-state extracts start arriving from several places at once.

10. Deliverables and how to run

Path	Contents
report/dashboard.html	Interactive 6-page dashboard replicating the Power BI report, plus a build walkthrough. Fully self-contained — opens in any browser with no dependencies.
report/...Submission.docx	This document.
excel/...Report.xlsx	10-sheet Excel pack: exec summary, merchant scorecard, voucher analysis, operational view, anomalies, ML results, business questions, data dictionary, assumptions.
dbt/	21 models, 132 tests, 5 seeds, 3 exposures, macros. Runs locally (dev) or Fabric Warehouse (prod).
sql/	Fabric Warehouse DDL and the business-question queries.
notebooks/	Fabric PySpark notebooks: bronze→silver, ML scoring with MLflow.
datafactory/	Pipeline JSON, schedule trigger, and the design rationale.
dax/	31+ DAX measures across 4 files, with commentary on each convention.
scripts/	Runnable reference implementation: profile, warehouse, analytics, ML, reconciliation, DAX validation, Excel, dashboard.
data/	bronze / silver / gold / analytics / ml layers as parquet, plus the local warehouse file.
docs/	Profile report, analytics summary, ML summary, reconciliation, DAX validation.

Reproducing the results

The full pipeline runs end to end in under two minutes:

```
python scripts/01_profile.py
python scripts/02_build_warehouse.py
python scripts/03_analytics.py
python scripts/04_ml_models.py
cd dbt && dbt seed && dbt run && dbt test
python scripts/05_reconcile.py
python scripts/06_validate_dax.py
python scripts/07_build_excel.py
python scripts/08_build_dashboard.py
```

A closing note on verification

Three claims in this submission are independently checkable rather than asserted. The gold layer is built twice by different engines and reconciles to the cent. Every DAX measure has a SQL-derived expected value in docs/dax_validation.csv. And the anomaly model recovered all four deliberately embedded patterns without being told what they were. Where a result was weak or a correlation was confounded, that is stated in the report rather than smoothed over — which is the more useful habit in a BI function, because the cost of a confident wrong number is far higher than the cost of an honest caveat.

Appendix F. End-to-end Fabric process captures

The following ten captures present the implemented process in operational order: the source ZIP on the PC, OneLake landing, deployed extraction code, extracted CSV files, Data Factory orchestration, Bronze, Silver, Gold, the final validation gate and a live reporting query.

1. Source ZIP on the PC

The supplied delivery is packaged as one ZIP containing the four expected source files. This is the immutable input uploaded to OneLake.

00

PC DELIVERY PACKAGE

Source archive before upload to Microsoft Fabric

C:\Users\Anthony.DESKTOP-E55HL78\Downloads\Merchant_Voucher_Intelligence\fabric_upload\landing\MerchantVoucher_Raw.zip

ARCHIVE MEMBER	BYTES
MerchantReference.csv	2,143
MerchantSales.csv	2,058,004
VoucherRedemptions.csv	9,050,258
SupportTickets.csv	127,372

Package size: 1,529,949 bytes | Members: 4

Retrieved 2026-08-11 22:39 SAST | Workspace 0f8b1362-fedb-4c4d-aecc-f788a989c6b2

Figure F.1 - Local PC delivery package and the four source members before upload.

2. PC package lands in OneLake

MerchantVoucher_Raw.zip was uploaded from the project PC to the Lakehouse landing area. The raw package is retained before extraction so the delivery can be replayed and audited.

01

ONELAKE LANDING
PC delivery package verified through the live OneLake DFS endpoint

MerchantVoucher_Raw.zip

AVAILABLE

OneLake path	b2e7b116-08a5-42cd-912d-5610f64c5489/Files/landing/MerchantVoucher_Raw.zip
Content length	1,529,949 bytes
Content type	application/octet-stream
Last modified	Tue, 11 Aug 2026 19:54:30 GMT
Etag	0x8DEF7E25BCC53C8
Next activity	nb_01_pc_zip_to_bronze

Control: raw ZIP retained for replay before extraction

Retrieved 2026-08-11 22:39 SAST | Workspace 0f8b1362-fedb-4cdd-aecc-f788a989c6b2

Figure F.2 - Live OneLake DFS verification of the uploaded package before processing.

3. Fabric notebook unpacks and ingests

The first deployed PySpark notebook expands the archive, reads each CSV through an absolute OneLake ABFSS path, adds lineage columns and writes raw Delta tables.

PC ZIP TO BRONZE | EXECUTED PYSPARK

Exact deployed extraction and Delta-write logic

```
1 zip_path = "/lakehouse/default/Files/landing/MerchantVoucher_Raw.zip"
2 extract_path = "/lakehouse/default/Files/landing/extracted"
3 Path(extract_path).mkdir(parents=True, exist_ok=True)
4
5 # The upload arrives as one immutable package from the analyst's PC. Unzip it
6 # inside OneLake so Bronze always has a replayable copy of the exact delivery.
7 with ZipFile(zip_path, "r") as archive:
8     archive.extractall(extract_path)
9     extracted = archive.namelist()
10 sources = {
11     "MerchantReference.csv": "bronze_merchant_reference",
12     "MerchantSales.csv": "bronze_merchant_sales",
13     "VoucherRedemptions.csv": "bronze_voucher_redemptions",
14     "SupportTickets.csv": "bronze_support_tickets",
15 }
16 onelake_extract_path = (
17     "abfss://WS_MerchantVoucher@onelake.dfs.fabric.microsoft.com/"
18     "LH_MerchantVoucher.Lakehouse/Files/landing/extracted"
19 )
20 run_id = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
21 for file_name, table_name in sources.items():
22     source = f"{onelake_extract_path}/{file_name}"
23     frame = (spark.read.option("header", True).option("inferSchema", True).csv(source)
24             .withColumn("_source_file", F.lit(file_name))
25             .withColumn("_run_id", F.lit(run_id))
26             .withColumn("_ingested_at_utc", F.current_timestamp()))
27     frame.write.format("delta").mode("overwrite").option(
28         "overwriteSchema", "true").saveAsTable(table_name)
29     print(f"BRONZE {table_name:<34} {frame.count():>9,} rows")
30
31 print("Bronze ingestion complete.")
```

Source: deployed notebook definition; pipeline run completed successfully

Figure F.3 - Exact deployed ZIP extraction, OneLake read and Bronze Delta-write logic.

4. Extracted source files verified in OneLake

After extraction, all four expected CSV files are independently verified in the Lakehouse Files area before table creation.

02

ONELAKE EXTRACTED FILES

Live DFS verification after nb_01_pc_zip_to_bronze extraction

Files/landing/extracted		
FILE	BYTES	
MerchantReference.csv	2,143	OK
MerchantSales.csv	2,058,004	OK
VoucherRedemptions.csv	9,050,258	OK
SupportTickets.csv	127,372	OK

Control: all four expected CSV members exist in OneLake after extraction

Retrieved 2026-08-11 22:39 SAST | Workspace 0f8b1362-fedb-4c4d-aecc-f788a989c6b2

Figure F.4 - Live OneLake verification of the four CSV files produced by extraction.

5. Data Factory executes the medallion chain

The pipeline runs ZIP-to-Bronze, Bronze-to-Silver, Silver-to-Gold and the final quality gate in dependency order. The retained run below completed without a failure reason.



Figure F.5 - Completed Microsoft Fabric Data Factory pipeline run 30f66673-261b-43cc-8d1b-58bf9516f3f0.

6. Bronze preserves the delivery

The first notebook extracts the four CSV files, reads them through their OneLake ABFSS paths and writes raw Delta tables with source-file, run-ID and ingestion-time lineage.

03

BRONZE LAYER
Live SELECT COUNT_BIG(*) results from LH_MerchantVoucher

TABLE	ROWS	
bronze_merchant_reference	25	OK
bronze_merchant_sales	26,500	OK
bronze_voucher_redemptions	120,969	OK
bronze_support_tickets	1,363	OK

Control: raw values plus source-file, run-ID and ingestion-time lineage

Retrieved 2026-08-11 22:39 SAST | Workspace 0f8b1362-fedb-4c4d-aecc-f788a989c6b2

Figure F.6 - Live Bronze table counts after PC ZIP extraction and raw Delta ingestion.

7. Silver conforms and cleans

The Silver notebook normalises column names, trims strings and removes duplicates using the declared business keys. Counts remain reconciled to Bronze because no valid source rows are discarded.

04

SILVER LAYER

Live SELECT COUNT_BIG(*) results from LH_MerchantVoucher

TABLE	ROWS
silver_merchant_reference	25 OK
silver_merchant_sales	26,500 OK
silver_voucher_redemptions	120,969 OK
silver_support_tickets	1,363 OK

Control: names normalised, strings trimmed and business keys deduplicated

Retrieved 2026-08-11 22:39 SAST | Workspace 0f8b1362-fedb-4c4d-aecc-f788a989c6b2

Figure F.7 - Live Silver table counts after cleansing, conformance and deduplication.

8. Gold publishes reporting tables

The Gold notebook publishes the merchant dimension and the three reporting facts used by downstream BI. Date keys and operational flags are derived here, after source preservation and conformance have completed.

05

GOLD LAYER

Live SELECT COUNT_BIG(*) results from LH_MerchantVoucher

TABLE	ROWS
gold_dim_merchant	25 OK
gold_fct_merchant_sales	26,500 OK
gold_fct_voucher_redemptions	120,969 OK
gold_fct_support_tickets	1,363 OK

Control: reporting-ready merchant dimension and three conformed facts

Retrieved 2026-08-11 22:39 SAST | Workspace 0f8b1362-fedb-4c4d-aecc-f788a989c6b2

Figure F.8 - Live Gold dimension and fact populations available to the reporting layer.

9. Validation gate reconciles all layers

The final notebook checks all twelve expected tables and fails the pipeline if any table is empty. The combined result below reconciles each business population across layers.

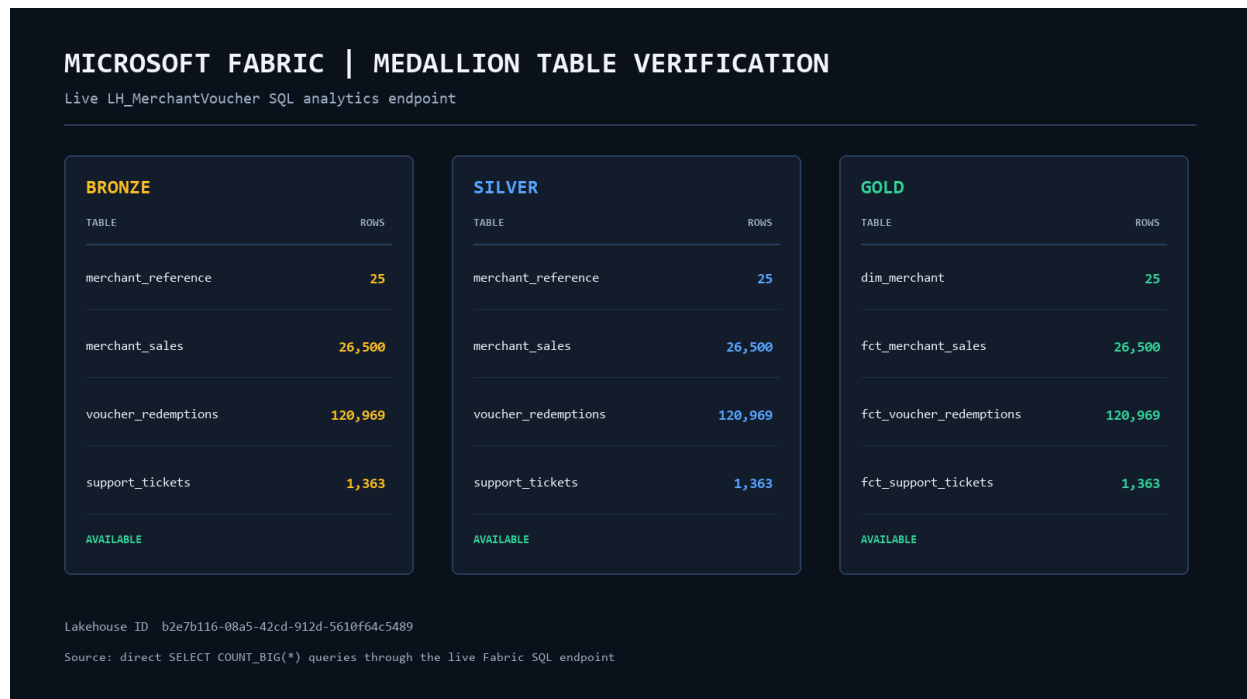


Figure F.9 - Live Bronze, Silver and Gold reconciliation after the quality gate.

10. Gold serves the reporting query

A Fabric-compatible T-SQL query runs against the reporting layer after publication. This closes the process by demonstrating that the curated output is queryable for BI.

Q1 - top merchants by value and volume

WH_MerchantVoucher | Live result from Fabric-compatible T-SQL 2026-08-11 21:26 SAST

MERCHANT_NAME	REGION	TOTAL_SALES	TOTAL_TRANSACTIONS	RANK_BY_VALUE	RANK_BY_VOLUME
Durban Cash Hub	Western Cape	R 5,776,118.74	45371	1	1
Kudu Digital Kiosk	KwaZulu-Natal	R 4,535,329.73	35458	2	3
Zebra Route Services	Gauteng	R 4,519,761.94	35910	3	2
Karoo Quick Shop	Gauteng	R 4,395,067.11	34657	4	4
Lighthouse General Dealer	Gauteng	R 3,965,086.39	31124	5	5
North Coast Connect	Free State	R 3,731,381.85	29341	6	6
Ubuntu Trading Post	KwaZulu-Natal	R 3,548,946.51	27865	7	7
Table Bay Express	Eastern Cape	R 3,325,306.26	26056	8	8
Blue Crane Trading	Gauteng	R 3,265,565.26	25532	9	9
Liberty Lane Stores	Western Cape	R 3,185,149.79	24828	10	10

10 row(s) returned

Figure F.10 - Live reporting result from the Fabric SQL endpoint: top merchants by sales value and transaction volume.

Capture scope. The OneLake, pipeline, Lakehouse and reporting frames are rendered from live Microsoft Fabric service responses. The source and code frames are rendered from the exact package and deployed notebook definition. They document the full executed process without pretending to be recreated Fabric portal interface screens.